ISPE 2025 Workshop

Introduction to **R Shiny for Pharmacoepidemiologists**: Interactive Visualization in Real-World Evidence Studies

Máté Szilcz, PhD
Yuchen Guo, PhD
Jyotirmoy Sarker, PhD candidate

Monday, August 25, 2025
1:00 PM – 2:30 PM

https://github.com/szilczmate/ISPE2025

Karolinska Institutet
Viti Science DATA ANALYTICS
University of Oxford
NDORMS Nuffield Department of Orthopaedics, Rheumatology and Musculoskeletal Sciences Medical Sciences Division
UIC University of Illinois Chicago

Disclaimer

Máté Szilcz, CEO & Founder of **Viti Science**, a consulting company specialized in R Shiny app developments.

Disclaimer

Máté Szilcz, CEO & Founder of **Viti Science**, a consulting company specialized in R Shiny app developments.

🤖 Parts of this presentation were created by **Artificial Intelligence** and may contain errors.

Disclaimer

Máté Szilcz, CEO & Founder of **Viti Science**, a consulting company specialized in R Shiny app developments.

🤖 Parts of this presentation were created by **Artificial Intelligence** and may contain errors.

🧠 Parts of this presentation were created by **Human Intelligence** and may also contain errors.


Mentimeter

Learning objectives

Understand the **key functionalities of R Shiny** and how it applies to pharmacoepidemiologic research.

Gain **practical experience** in developing a basic interactive visualization tool using R Shiny.

Learn how to create **user-friendly interfaces and server logic** to enable interactive data exploration.

Explore best practices for using **R Shiny to enhance stakeholder engagement** and decision-making.

Build confidence in implementing R Shiny for research projects through guided exercises.

Workshop overview

Main goals
1. Introduction to R Shiny (20 min) 2. Learn fundamentals of R Shiny app development (10 min) 3. Build a simple app to explore and visualize data (30 min) 4. Enhance interactivity (15 min) 5. Q&A (15 min)

Basic familiarity with R is recommended but not mandatory.
Follow-along coding with codebase provided.



Why learn about interactive visualization?

Relevance

Studies increasingly require real-time insights

Timeliness

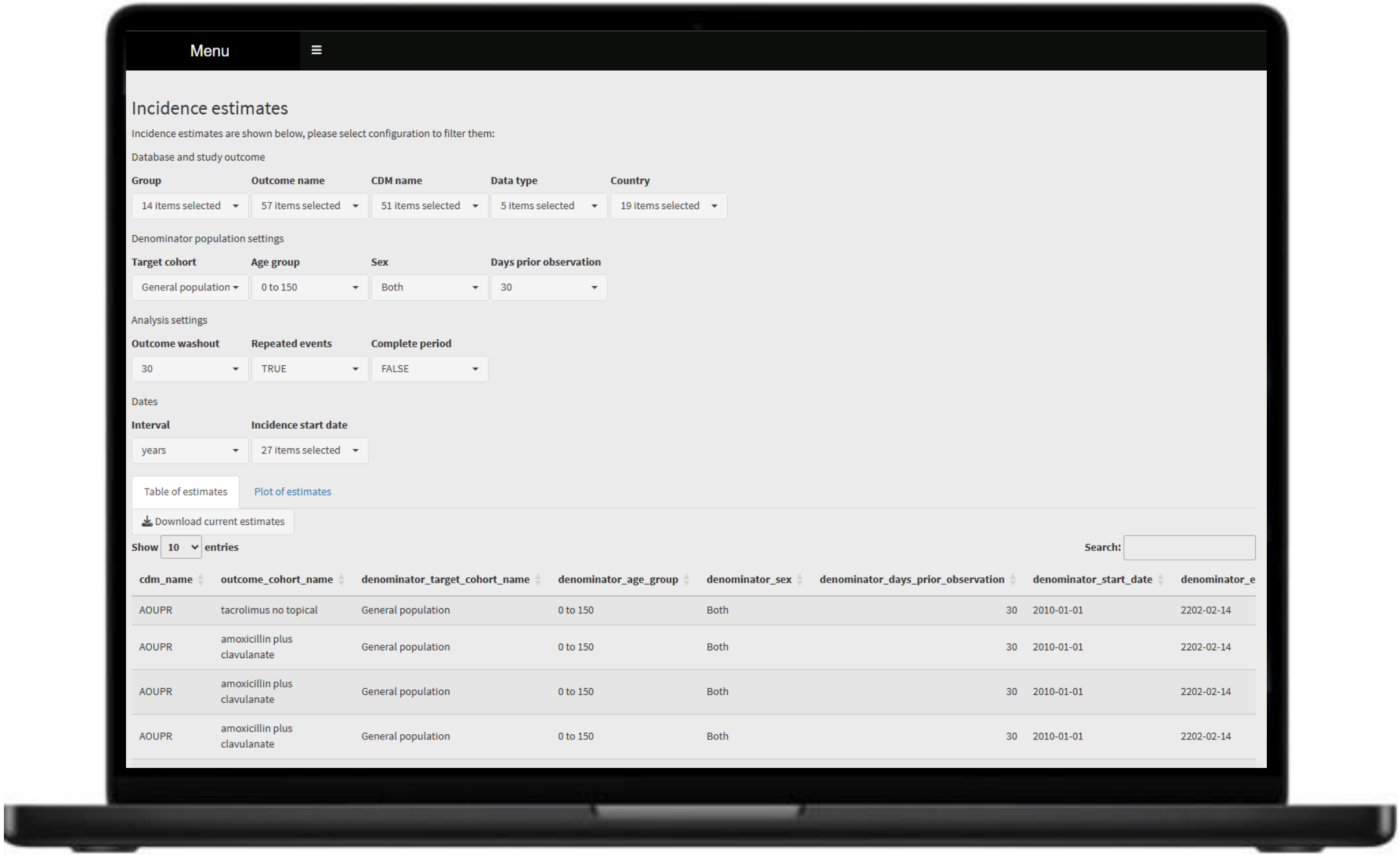
Growing use in both academia and industry

Uniqueness

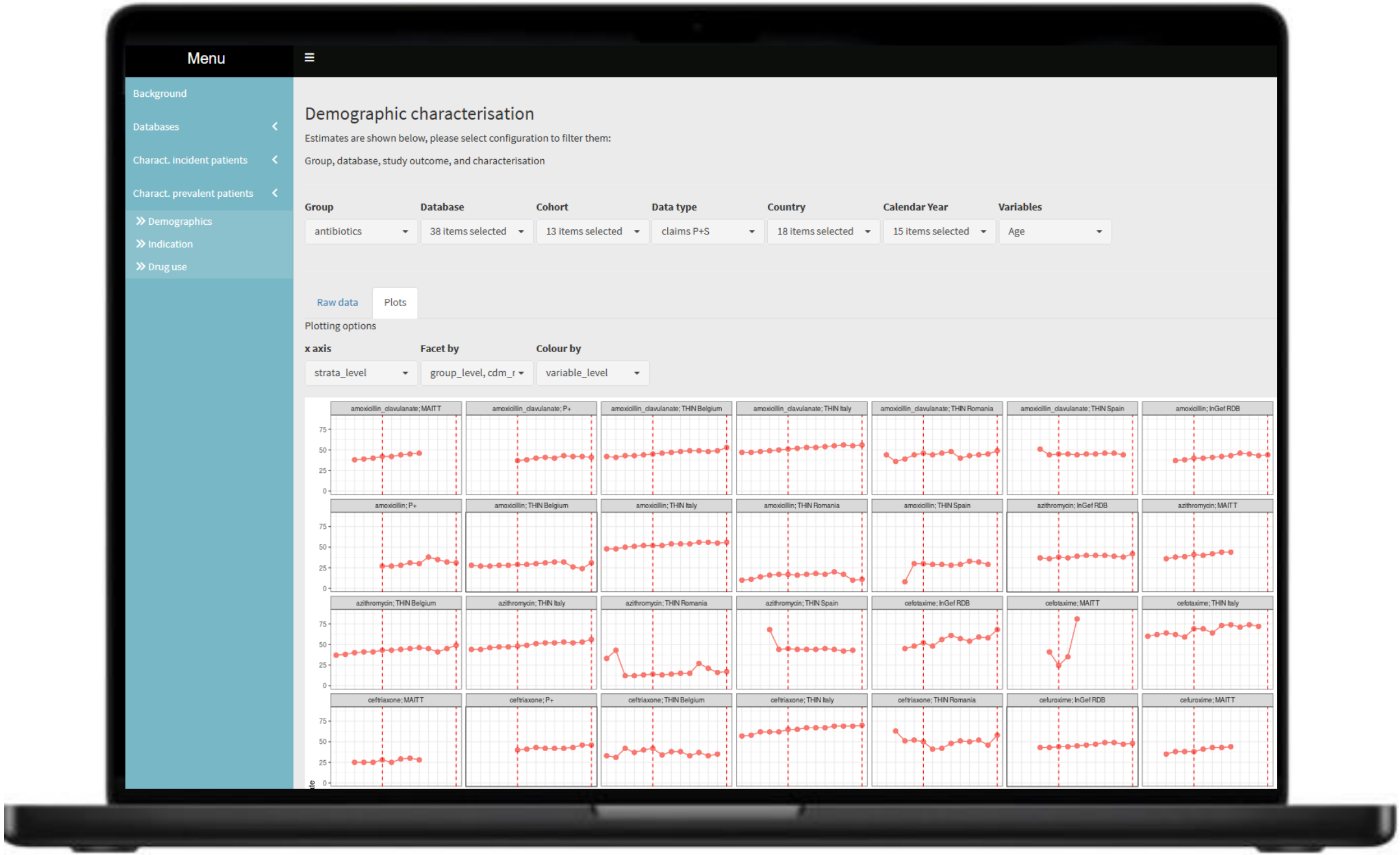
Tailored for pharmacoepi, not data science



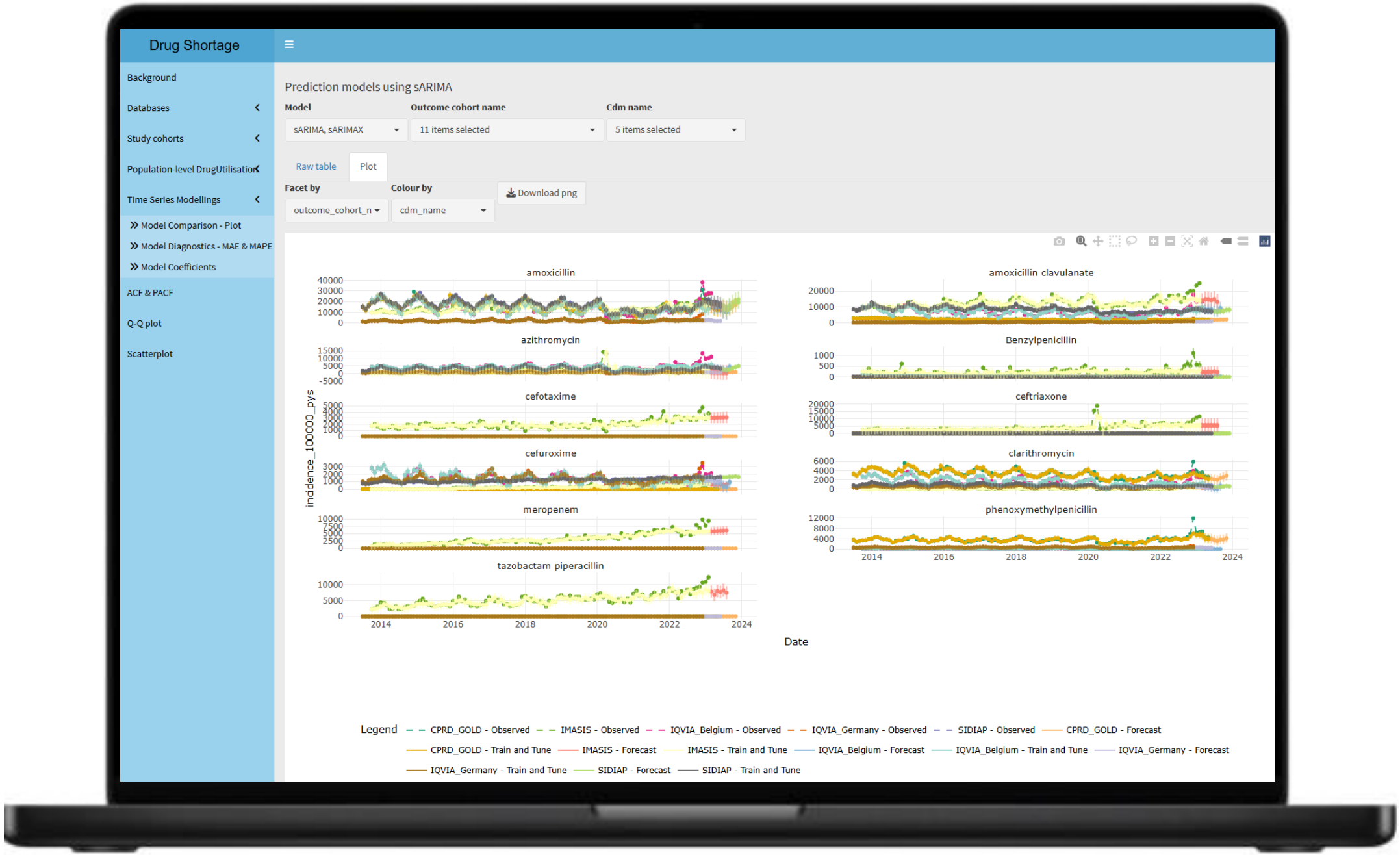
https://data.ehden.eu/MegaStudy_IncidencePrevalence/



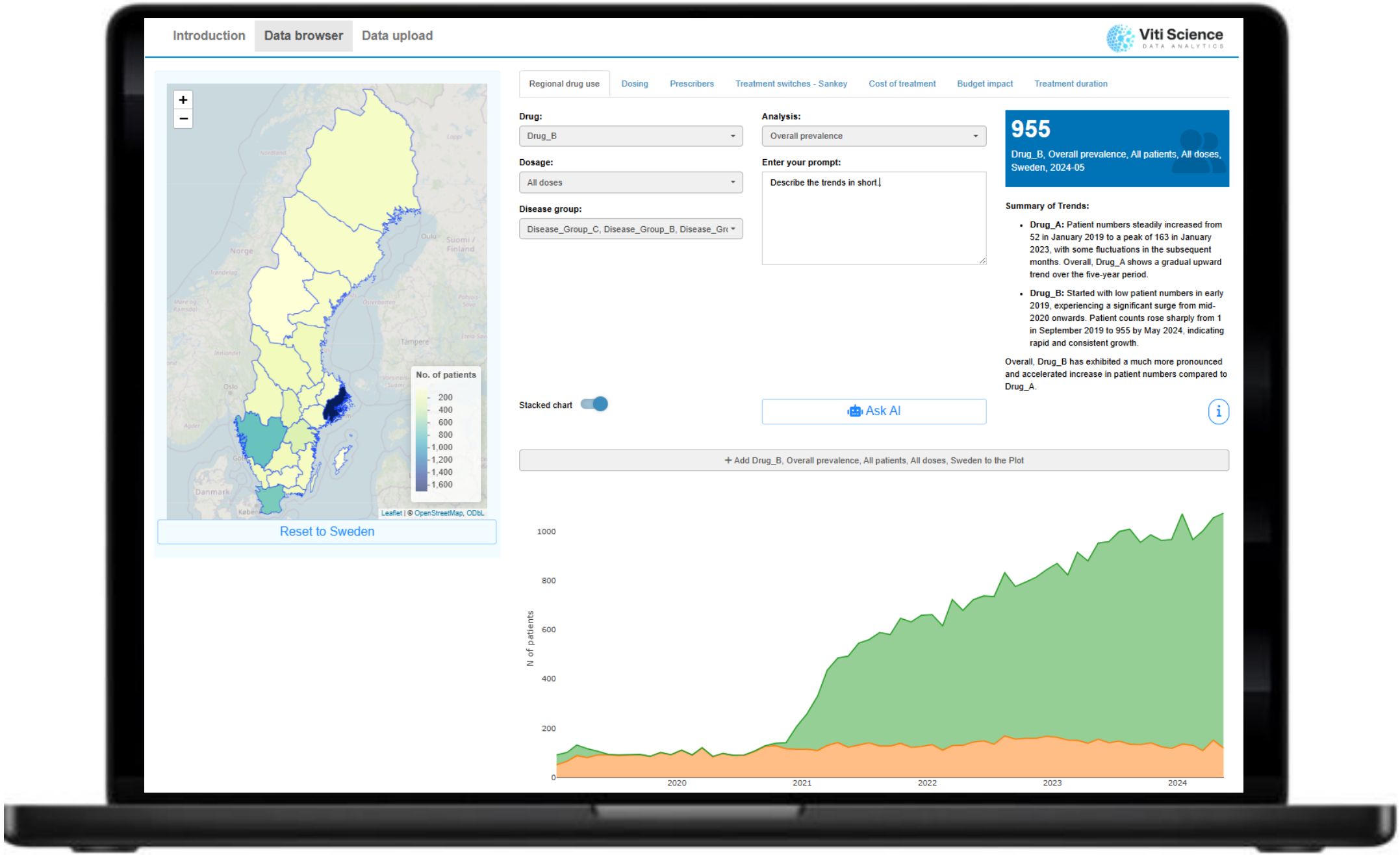
https://data.ehden.eu/MegaStudy_Dus/



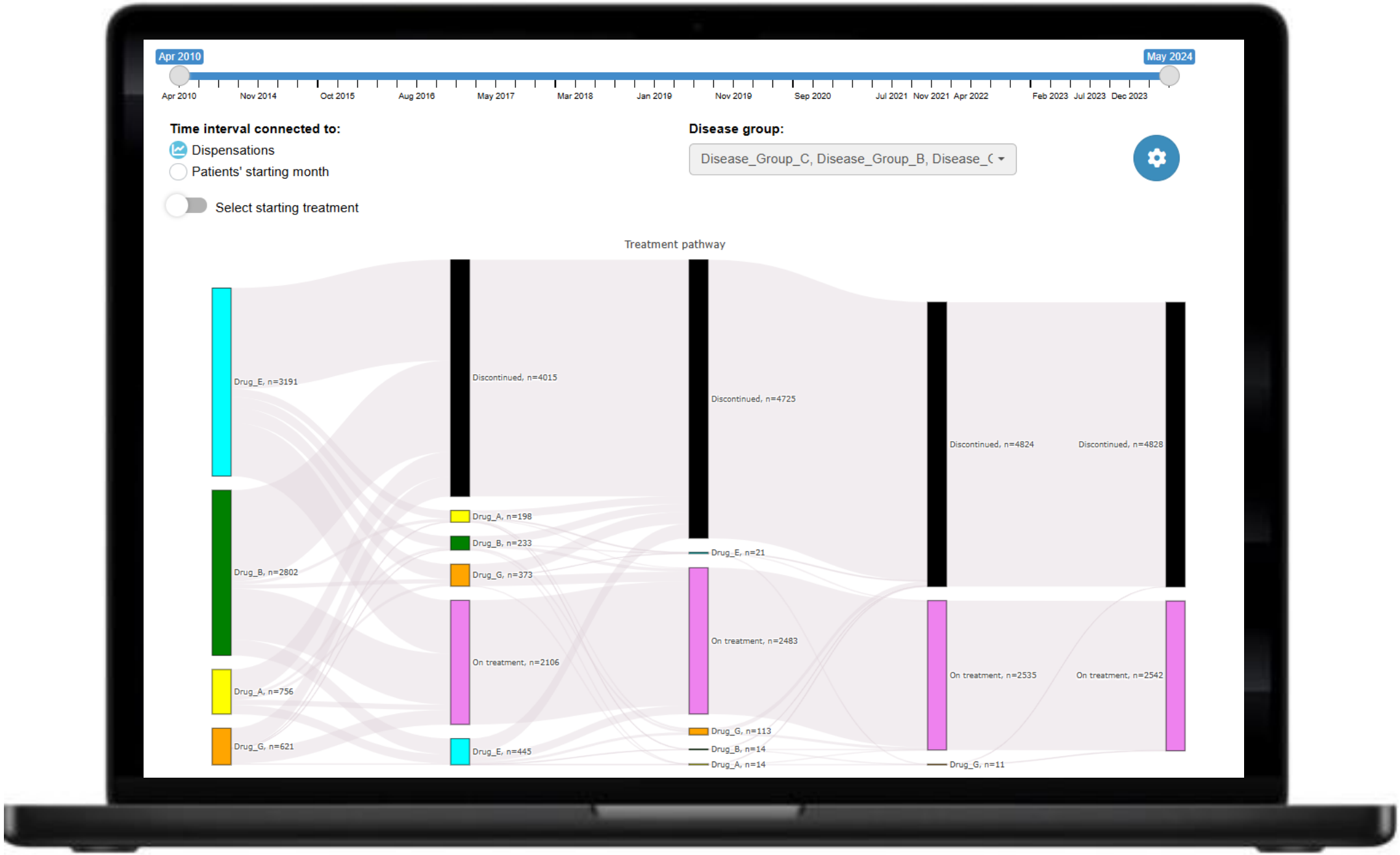
https://dpa-pde-oxford.shinyapps.io/P2-C1-012-DrugShortages/



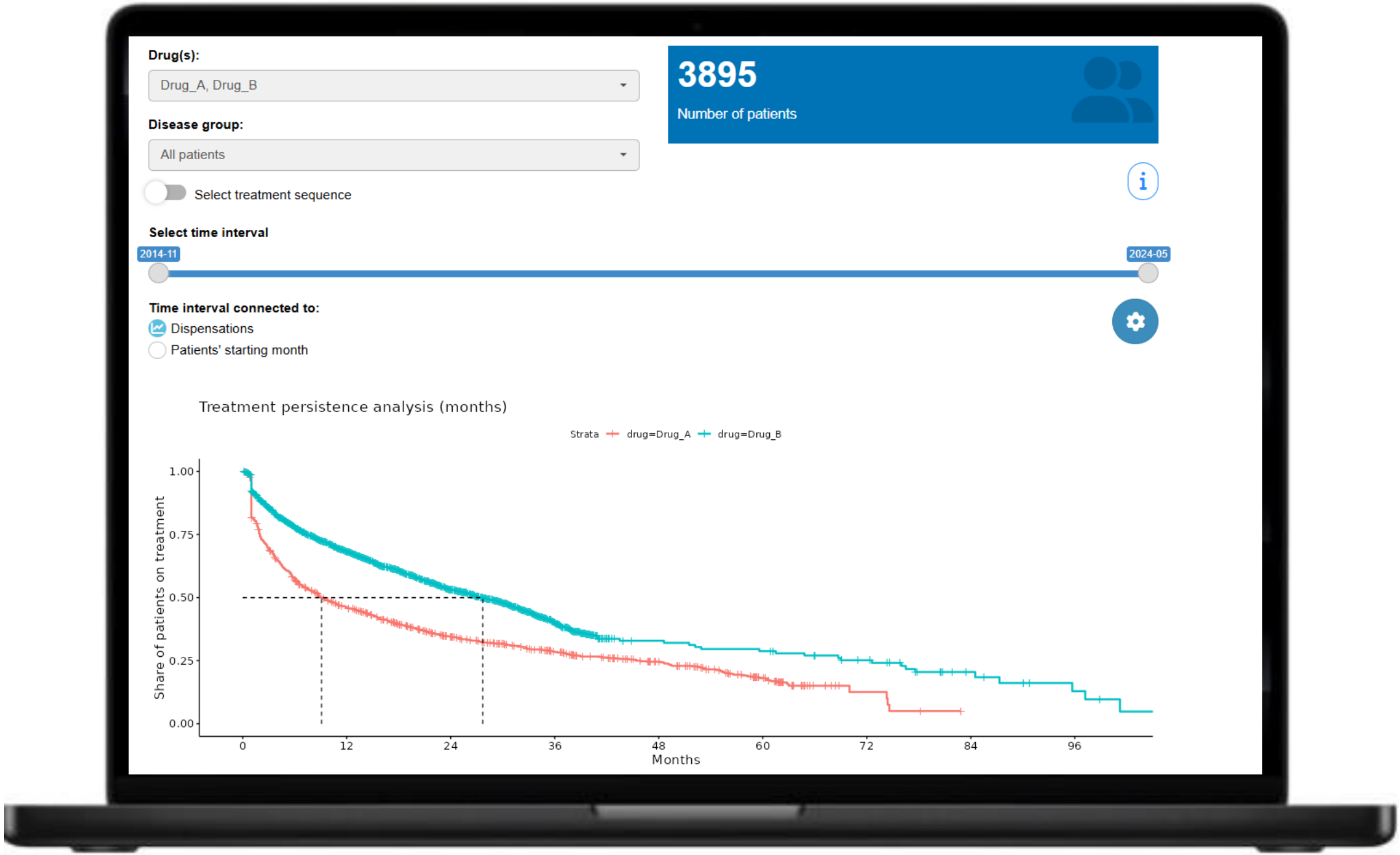
https://vitiscience.se/



https://vitiscience.se/



https://vitiscience.se/



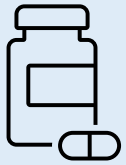
Enhance communication and transparency with R Shiny in pharmacoepi

In RWE studies, **clear and timely communication** of results is essential

Clinical & safety decisions



Regulatory discussions



Market access strategies



Public health



Enhance communication and transparency with R Shiny in pharmacoepi

Interactive Visualizations

Stakeholders can explore cohort definitions, outcomes, and sensitivity analyses hands-on

Real-Time Insights

Data updates instantly across views, no need for static reports.

Tailored Perspectives

Clinicians, epidemiologists, and decision-makers can interact with the data relevant to them.

Improved Transparency

Underlying assumptions, data sources, and methods can be embedded directly into the app.

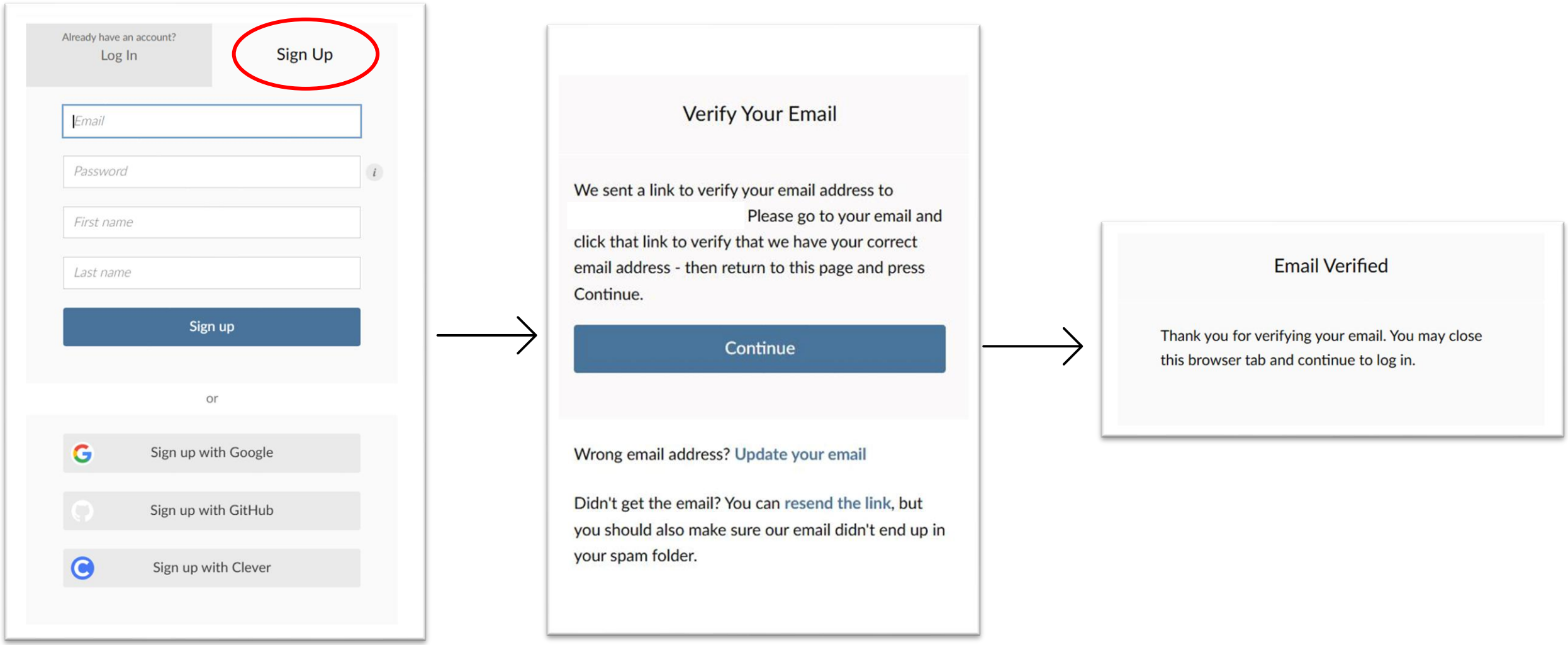


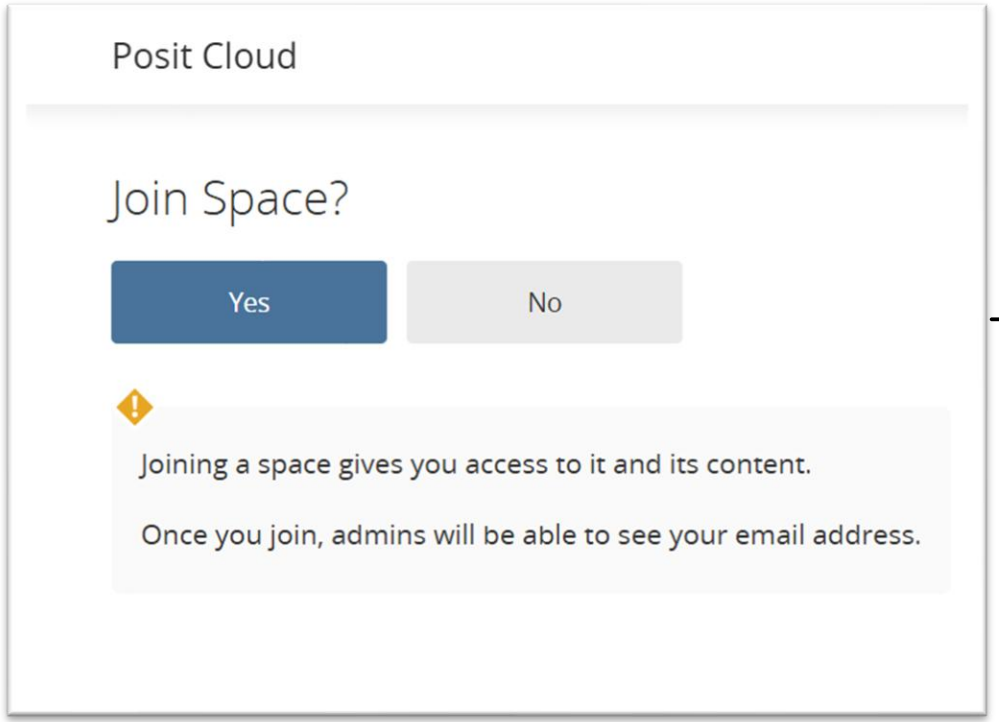
Follow-along coding via Posit Cloud

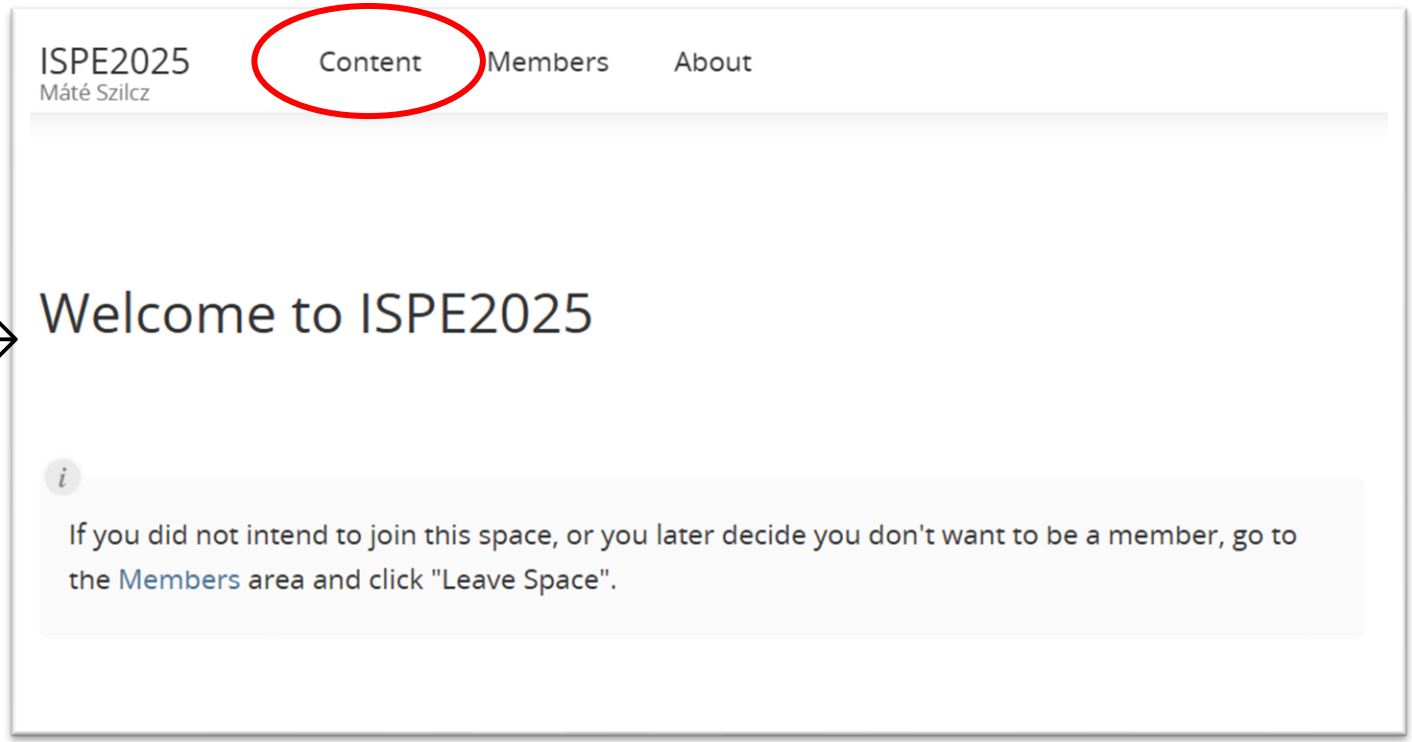
Scan the QR code or click the link below to open the workspace.

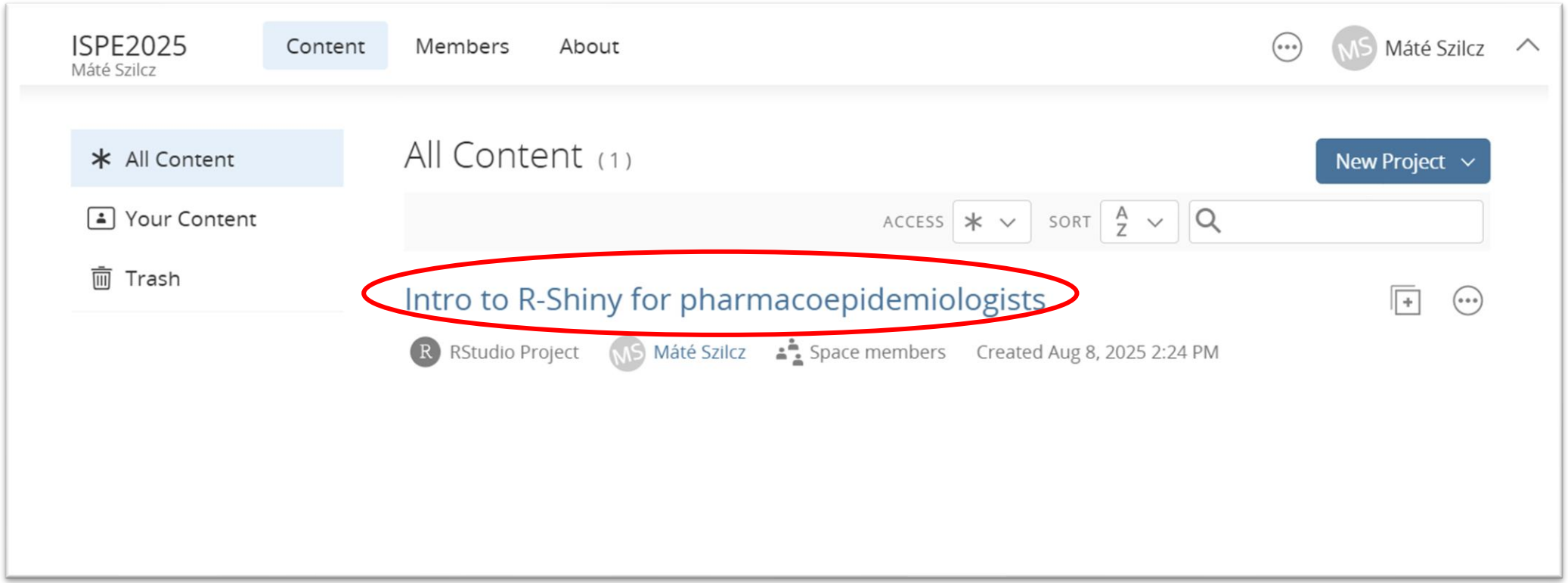
Registration required!

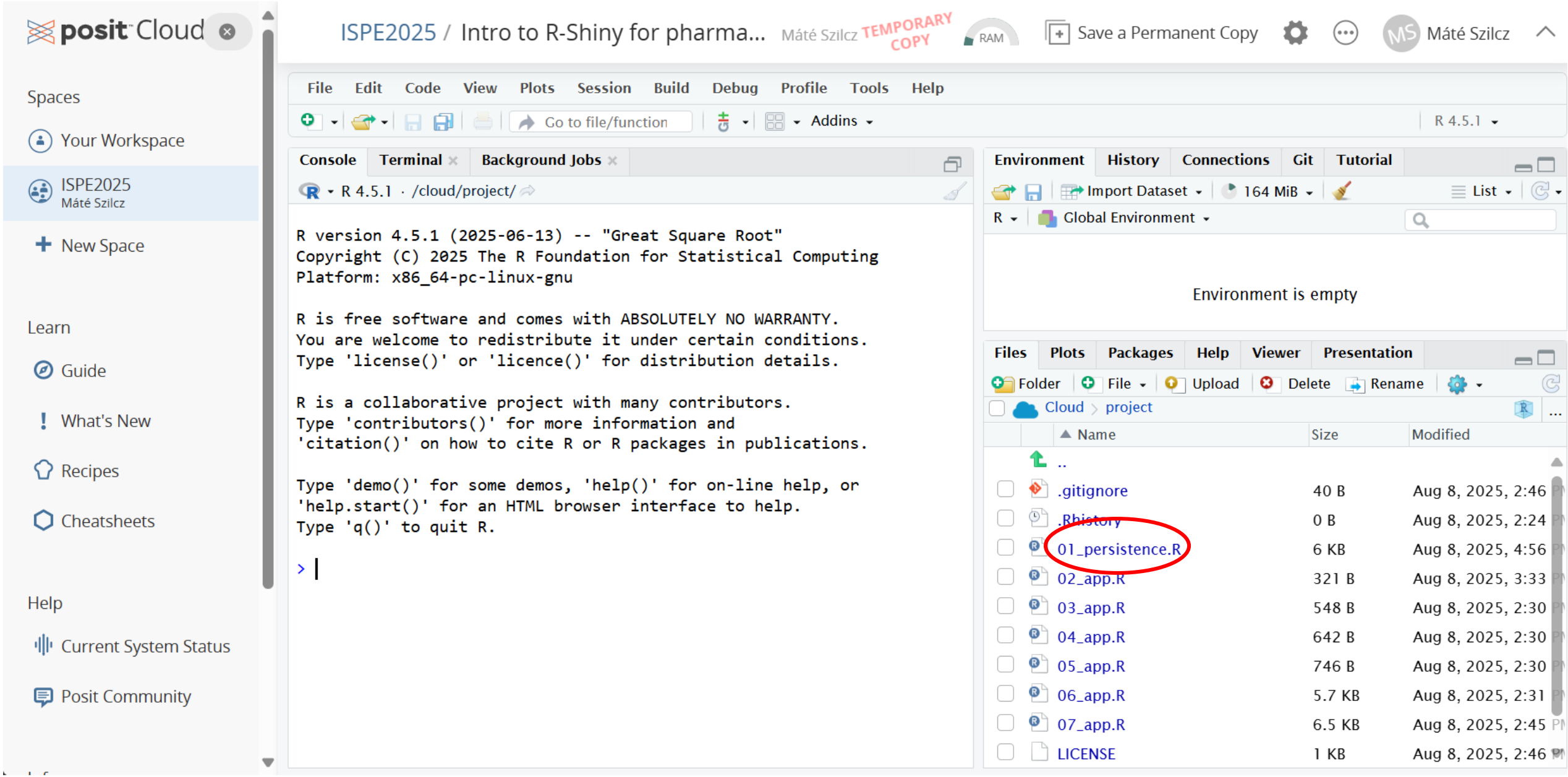
Or Click this link!


Already have an account?
Log In
Sign Up
Email
Password
First name
Last name
Sign up
or
Sign up with Google
Sign up with GitHub
Sign up with Clever
Verify Your Email
We sent a link to verify your email address to
Please go to your email and click that link to verify that we have your correct email address - then return to this page and press Continue.
Continue
Wrong email address? Update your email
Didn't get the email? You can resend the link, but you should also make sure our email didn't end up in your spam folder.
Email Verified
Thank you for verifying your email. You may close this browser tab and continue to log in.


Posit Cloud
Join Space?
Yes
No
Joining a space gives you access to it and its content.
Once you join, admins will be able to see your email address.


ISPE2025
Máté Szilcz
Content
Members
About
Welcome to ISPE2025
If you did not intend to join this space, or you later decide you don't want to be a member, go to the Members area and click "Leave Space".


ISPE2025
Máté Szilcz
Content
Members
About
MS
Máté Szilcz
All Content
Your Content
Trash
All Content (1)
New Project
ACCESS
SORT
Intro to R-Shiny for pharmacoepidemiologists
R
RStudio Project
MS
Máté Szilcz
Space members
Created Aug 8, 2025 2:24 PM


posit Cloud
Spaces
Your Workspace
ISPE2025
Máté Szilcz
New Space
Learn
Guide
What's New
Recipes
Cheatsheets
Help
Current System Status
Posit Community
ISPE2025 / Intro to R-Shiny for pharma...
Máté Szilcz TEMPORARY COPY
RAM
Save a Permanent Copy
Máté Szilcz
File Edit Code View Plots Session Build Debug Profile Tools Help
Go to file/function
Addins
R 4.5.1
Console
Terminal
Background Jobs
R 4.5.1 · /cloud/project/
R version 4.5.1 (2025-06-13) -- "Great Square Root"
Copyright (C) 2025 The R Foundation for Statistical Computing
Platform: x86_64-pc-linux-gnu
R is free software and comes with ABSOLUTELY NO WARRANTY.
You are welcome to redistribute it under certain conditions.
Type 'license()' or 'licence()' for distribution details.
R is a collaborative project with many contributors.
Type 'contributors()' for more information and
'citation()' on how to cite R or R packages in publications.
Type 'demo()' for some demos, 'help()' for on-line help, or
'help.start()' for an HTML browser interface to help.
Type 'q()' to quit R.
>
Environment
History
Connections
Git
Tutorial
Import Dataset
164 MiB
List
R
Global Environment
Environment is empty
Files
Plots
Packages
Help
Viewer
Presentation
Folder
File
Upload
Delete
Rename
Cloud
project
Name
Size
Modified
..
.gitignore 40 B Aug 8, 2025, 2:46
.Rhistory 0 B Aug 8, 2025, 2:24
01_persistence.R 6 KB Aug 8, 2025, 4:56
02_app.R 321 B Aug 8, 2025, 3:33
03_app.R 548 B Aug 8, 2025, 2:30
04_app.R 642 B Aug 8, 2025, 2:30
05_app.R 746 B Aug 8, 2025, 2:30
06_app.R 5.7 KB Aug 8, 2025, 2:31
07_app.R 6.5 KB Aug 8, 2025, 2:45
LICENSE 1 KB Aug 8, 2025, 2:46

Case study

Let's run **persistence analysis** (time to first discontinuation) **in R**.

We have a **drug dispensation data**

- Drug A and B
- Both 168, 60, 100 unit per packs
- For indication 2 units per day
- 2 covariates: sex and age

```
> data <- generate_data()
> head(data,10)
   person_id  disp_date drug num_pack units_pack ddd_pack  sex age_group
1      P0001 2021-03-03    A        3        168       50 Male     45-64
2      P0001 2021-03-11    A        1        168       84 Male     45-64
3      P0001 2021-03-26    A        1        168       42 Male     45-64
4      P0001 2023-11-30    A        3        100       50 Male     45-64
5      P0001 2024-02-07    A        1        100       84 Male     45-64
6      P0001 2024-04-28    A        1        100       15 Male     45-64
7      P0001 2020-09-02    B        2        168       42 Male     45-64
8      P0001 2021-11-06    B        2        100       15 Male     45-64
9      P0001 2021-12-31    B        1        100       84 Male     45-64
10     P0001 2024-12-31    B        1        168       15 Male     45-64
>
```

File: 01_persistence.R in GitHub repository

Step 1

Data management

Ensure 1 row per person & drug & dispensation

Calculate end date based on a 90 day grace period

```
> data <- generate_data()
> head(data,10)
   person_id  disp_date drug num_pack units_pack ddd_pack  sex age_group
1      P0001 2021-03-03    A        3        168       50 Male     45-64
2      P0001 2021-03-11    A        1        168       84 Male     45-64
3      P0001 2021-03-26    A        1        168       42 Male     45-64
4      P0001 2023-11-30    A        3        100       50 Male     45-64
5      P0001 2024-02-07    A        1        100       84 Male     45-64
6      P0001 2024-04-28    A        1        100       15 Male     45-64
7      P0001 2020-09-02    B        2        168       42 Male     45-64
8      P0001 2021-11-06    B        2        100       15 Male     45-64
9      P0001 2021-12-31    B        1        100       84 Male     45-64
10     P0001 2024-12-31    B        1        168       15 Male     45-64
>
```

```r
# Aggregate within person_id, drug & date, summing DDDs and tablets
agg <- data |>
  dplyr::arrange(person_id, drug, disp_date) |>
  dplyr::group_by(person_id, drug, disp_date) |>
  dplyr::summarize(
    total_ddd = base::sum(ddd_pack * num_pack),
    total_tab = base::sum(units_pack * num_pack),
    sex       = dplyr::first(sex),
    age_group = dplyr::first(age_group),
    .groups   = "drop"
  )

# Calculate assumed end_date (2 units/day + max gap 90 days)
agg <- agg |>
  dplyr::mutate(
    days_supply = total_tab / 2,
    end_date    = disp_date + base::ceiling(days_supply) + lubridate::days(90)
  )
```

Step 2: prep data for survival analysis

```r
# Collapse into continuous episodes by drug
episodes <- agg |>
  dplyr::group_by(person_id, drug) |>
  dplyr::arrange(disp_date) |>
  dplyr::mutate(
    overlap = disp_date <= dplyr::lag(end_date),
    epi_id  = base::cumsum(dplyr::coalesce(!overlap, TRUE))
  ) |>
  dplyr::group_by(person_id, drug, epi_id) |>
  dplyr::summarize(
    start_date     = base::min(disp_date),
    end_date       = base::max(end_date),
    sex            = dplyr::first(sex),
    age_group      = dplyr::first(age_group),
    .groups        = "drop"
  ) |>
  dplyr::arrange(person_id, start_date)

# First episode for incident users
episodes <- episodes |>
  dplyr::filter(start_date >= as.Date("2020-01-01")) |>
  dplyr::group_by(person_id) |>
  dplyr::arrange(start_date) |>
  dplyr::filter(start_date == dplyr::first(start_date)) |>
  dplyr::ungroup()

# Censoring date and discontinuation indicator
study_end <- as.Date("2024-12-31")
episodes <- episodes |>
  dplyr::mutate(
    censor_date = base::pmin(end_date, study_end),
    disc        = dplyr::if_else(end_date < study_end, 1L, 0L),
    time_months  = round(lubridate::time_length(lubridate::interval(start_date, censor_date), unit = "month"),1)
  )
```

```
> head(episodes)
# A tibble: 6 × 10
  person_id drug  epi_id start_date end_date   sex    age_group censor_date  disc time_months
  <chr>     <chr>  <int> <date>     <date>     <chr>  <chr>     <date>       <int>       <dbl>
1 P0001     B          1 2020-09-02 2021-05-18 Male   45-64     2021-05-18       1         8.5
2 P0002     A          1 2020-01-15 2020-09-11 Male   65+       2020-09-11       1         7.9
3 P0003     B          1 2020-02-24 2020-08-22 Male   65+       2020-08-22       1         5.9
4 P0004     A          1 2020-01-03 2021-02-12 Female 45-64     2021-02-12       1        13.3
5 P0005     A          1 2020-05-23 2021-11-03 Male   65+       2021-11-03       1        17.4
6 P0006     A          1 2020-01-17 2020-10-07 Female 45-64     2020-10-07       1         8.7
>
```

One row per person (first treatment episode selected)

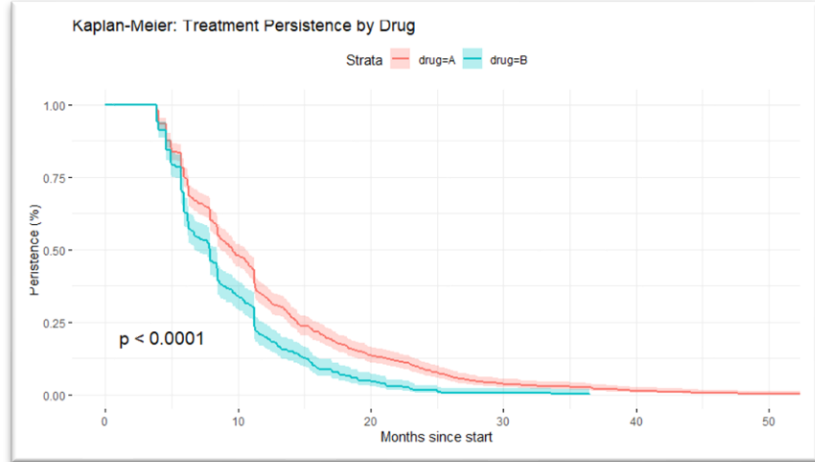
Event indicator variable: **disc (1 = discontinued)**

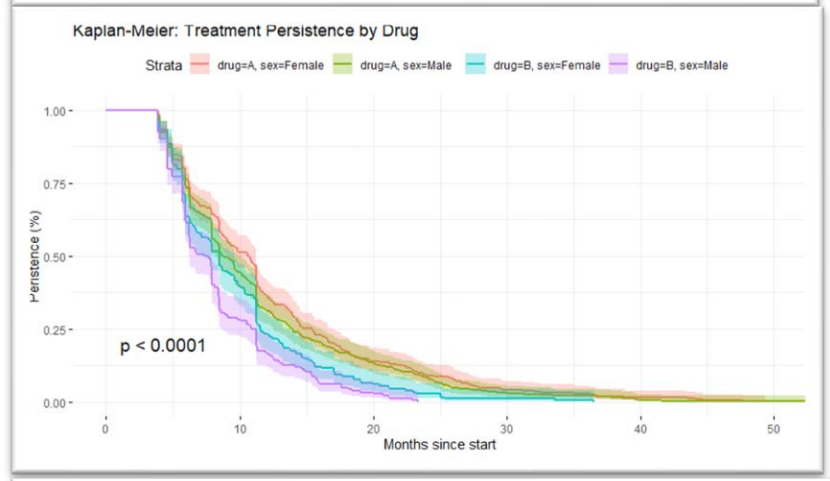
Time on treatment variable: **time_months**

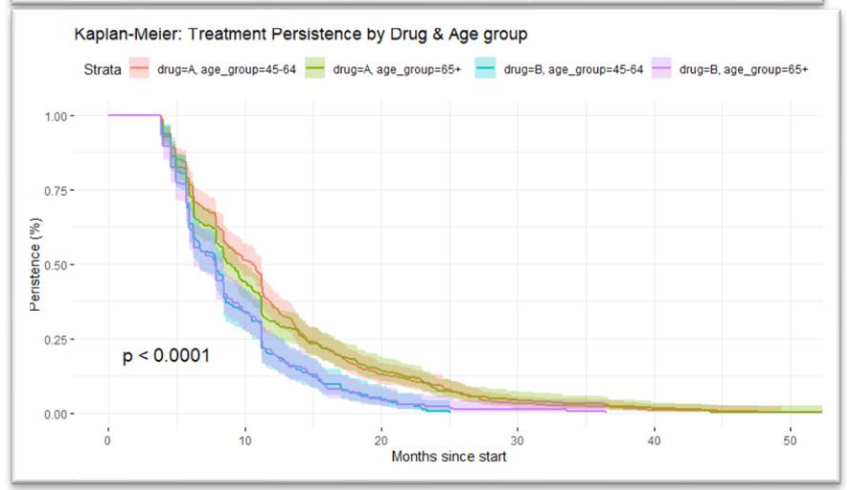
```r
# Plot persistence
fit <- survival::survfit(survival::Surv(time_months, disc) ~ drug, data = episodes)
survminer::ggsurvplot(
  fit,
  data = episodes,
  risk.table = TRUE,
  pval       = TRUE,
  conf.int   = TRUE,
  ggtheme    = ggplot2::theme_minimal(),
  title      = "Kaplan-Meier: Treatment Persistence by Drug",
  xlab       = "Months since start",
  ylab       = "Peristence (%)"
)

fit <- survival::survfit(survival::Surv(time_months, disc) ~ drug + sex, data = episodes)
survminer::ggsurvplot(
  fit,
  data = episodes,
  risk.table = TRUE,
  pval       = TRUE,
  conf.int   = TRUE,
  ggtheme    = ggplot2::theme_minimal(),
  title      = "Kaplan-Meier: Treatment Persistence by Drug & Sex",
  xlab       = "Months since start",
  ylab       = "Peristence (%)"
)

fit <- survival::survfit(survival::Surv(time_months, disc) ~ drug + age_group, data = episodes)
survminer::ggsurvplot(
  fit,
  data = episodes,
  risk.table = TRUE,
  pval       = TRUE,
  conf.int   = TRUE,
  ggtheme    = ggplot2::theme_minimal(),
  title      = "Kaplan-Meier: Treatment Persistence by Drug & Age group",
  xlab       = "Months since start",
  ylab       = "Peristence (%)"
)
```







Challenges with static R Code vs R Shiny

Static R Script Approach	R Shiny Advantage
Rigid outputs Plots are hard-coded and must be re-run for each subgroup (e.g., sex, age).	**Interactive filtering** Users can dynamically select drugs, sex, age groups via UI
No real-time exploration Analysts must rerun and regenerate plots for every analytical question.	**Exploratory workflow** Users can explore survival by subgroup, change cutoffs, or time windows instantly.
Hardcoded parameters Assumptions (e.g., 2 tablets/day, 90-day gap) are fixed in code.	**Configurable parameters** Users can adjust assumptions via sliders or inputs.
Output overload Multiple static plots clutter reports and are hard to compare side-by-side.	**Dynamic UI** Tabs, drop-downs, and responsive plots reduce clutter.
Limited stakeholder engagement Lengthy report (e.g., sensitivity analyses)	**Accessible to non-programmers** Stakeholders can self-serve and explore without coding.

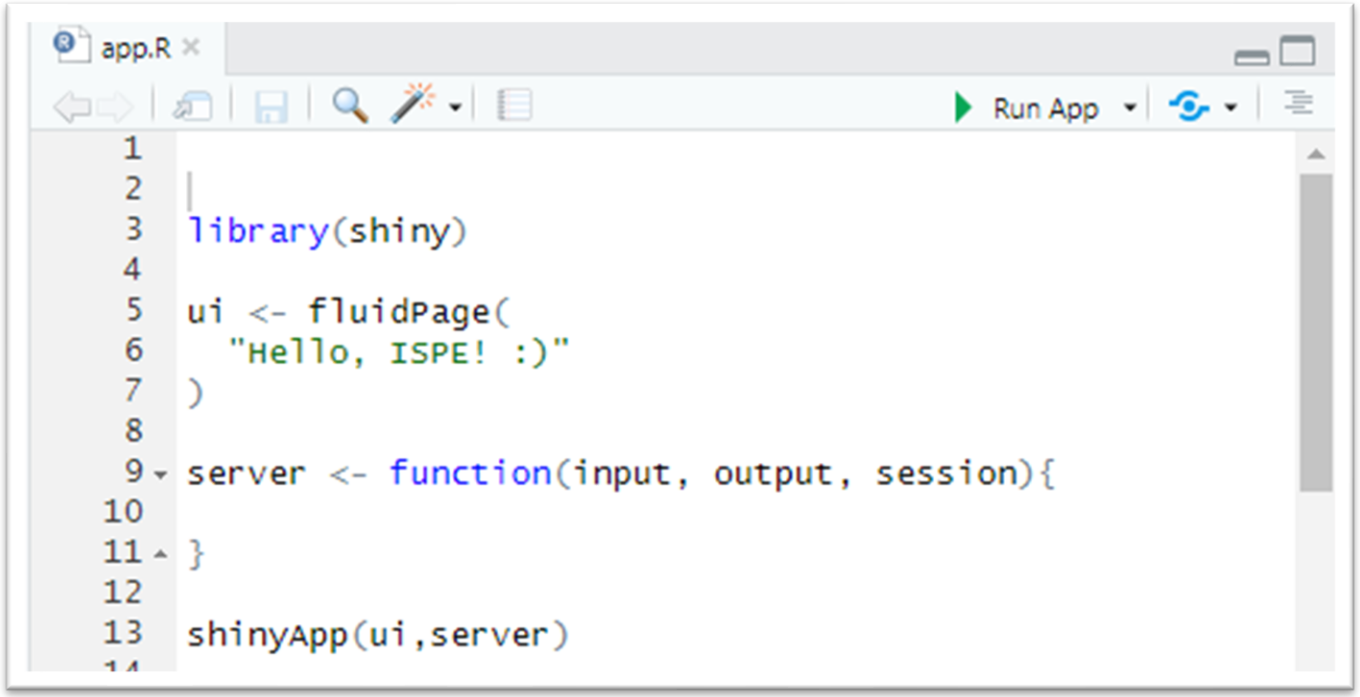
The basics

RStudio IDE

Shiny app has two parts:

- **UI**: User interface
- **Server**: server logic

One script: **app.R**



File: 02_app.R in GitHub repository

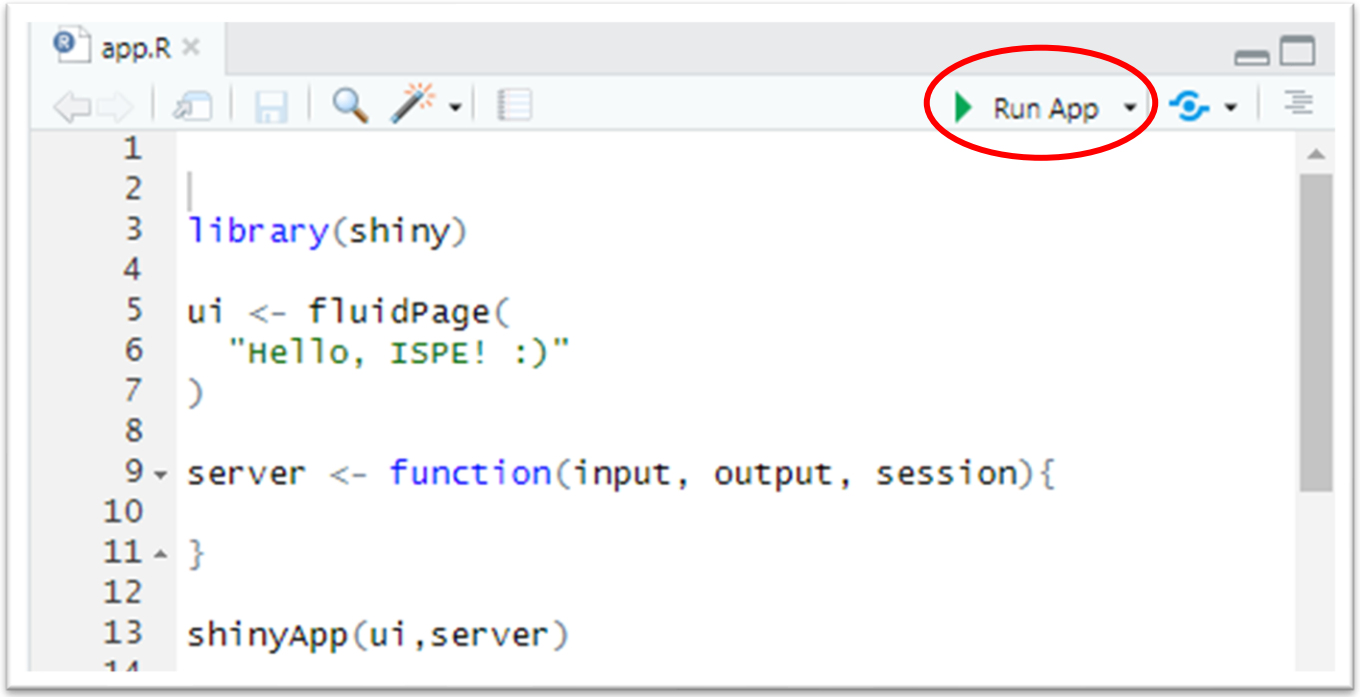
The basics

RStudio IDE

Shiny app has two parts:

- **UI**: User interface
- **Server**: server logic

One script: **app.R**



File: 02_app.R in GitHub repository

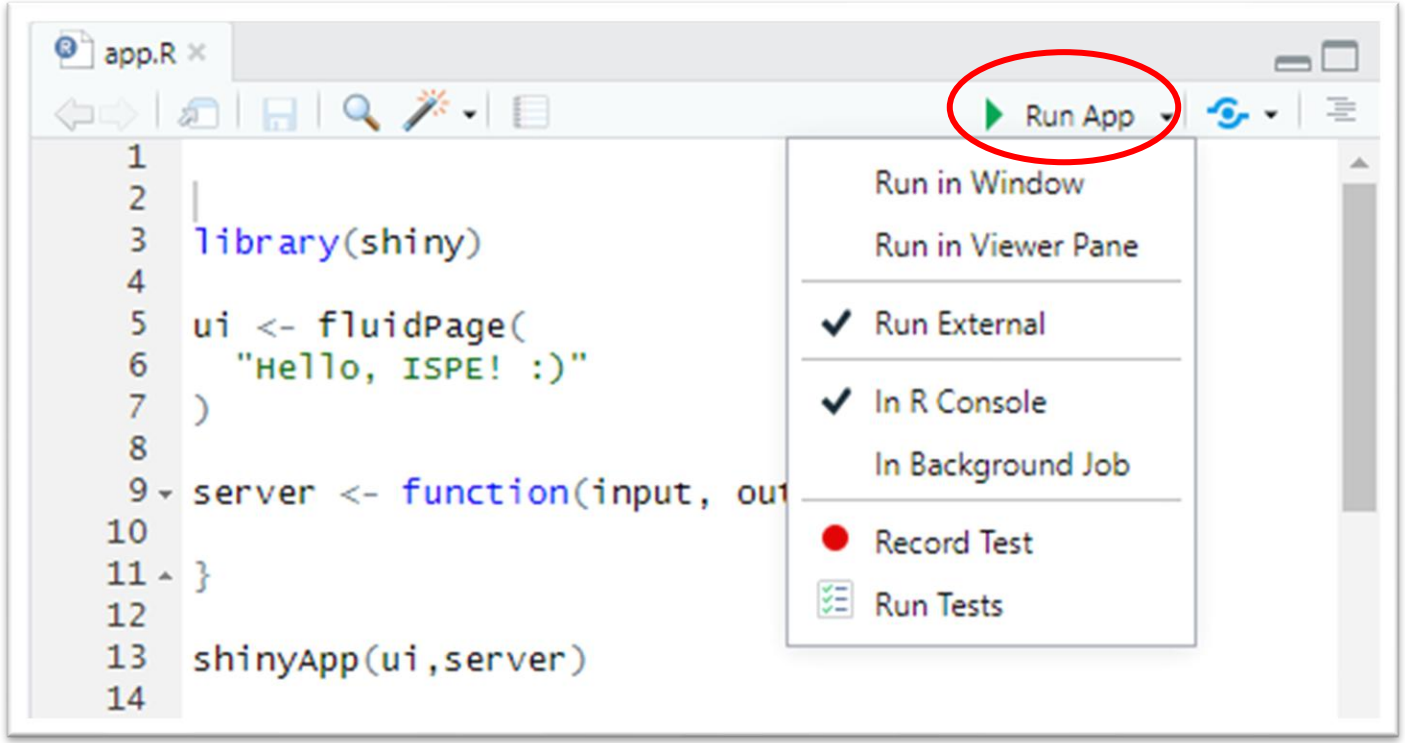
The basics

RStudio IDE

Shiny app has two parts:

- **UI**: User interface
- **Server**: server logic

One script: **app.R**



File: 02_app.R in GitHub repository

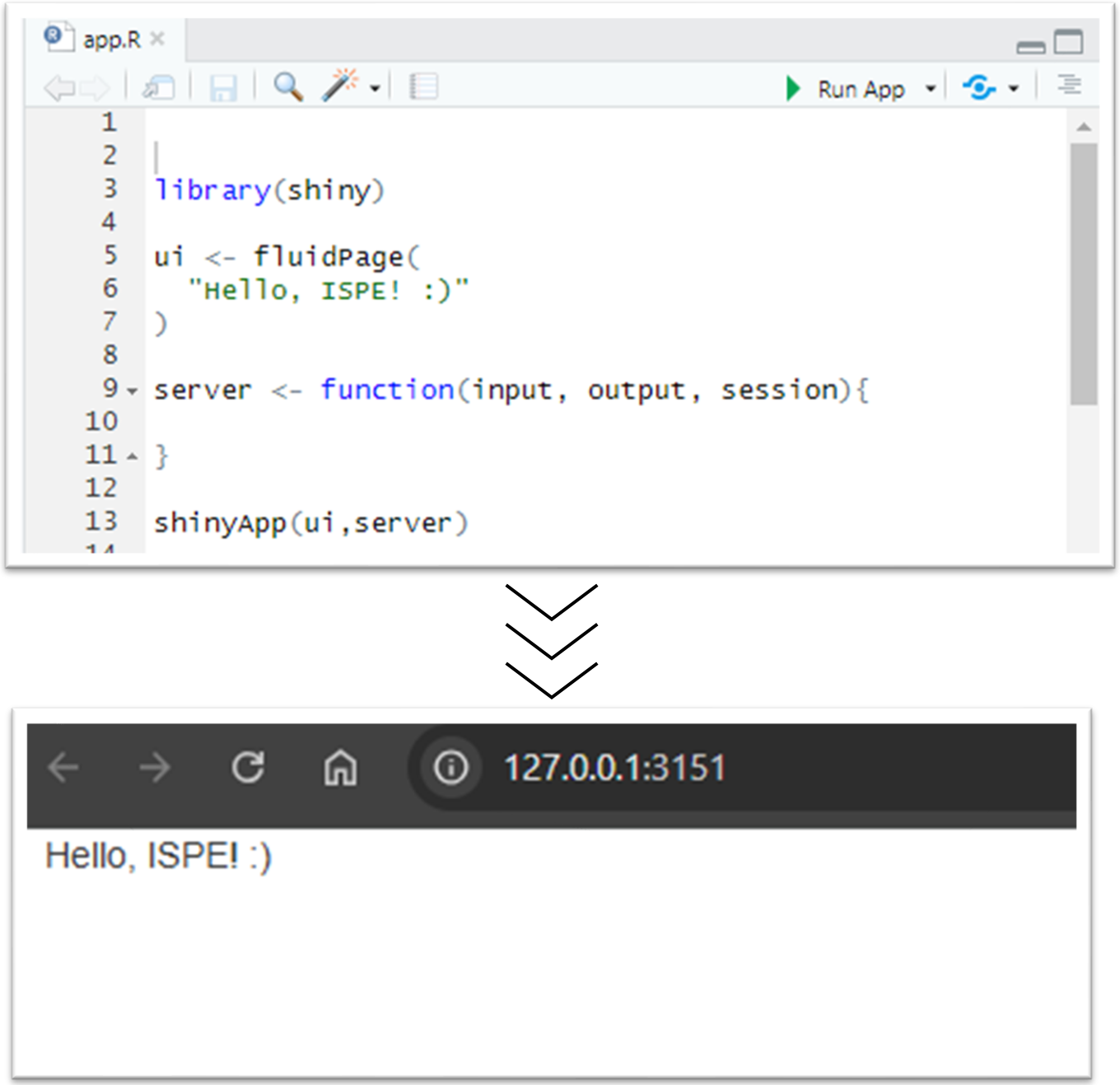
The basics

RStudio IDE

Shiny app has two parts:

- **UI**: User interface
- **Server**: server logic

One script: **app.R**



app.R

```
1
2  library(shiny)
3
4  ui <- fluidPage(
5    "Hello, ISPE! :)"
6  )
7
8  server <- function(input, output, session){
9
10 }
11
12 shinyApp(ui,server)
13
```

Loading the *shiny* package

If not available run *install.packages*(“shiny”)

app.R

```
1
2  library(shiny)
3
4  ui <- fluidPage(
5    "Hello, ISPE! :)"
6  )
7
8  server <- function(input, output, session){
9
10 }
11
12 shinyApp(ui,server)
13
```

fluidPage() is a wrapper of all the UI content.
It creates a **bootstrap** page.

Bootstrap

A popular and free **front-end framework** used to create responsive, modern user interfaces in web development

Built on

- **HTML** & **CSS** (make it pretty)
- **JavaScript** (make it do stuff)

Helps make Shiny apps look clean **without any knowledge** of HTML, CSS or JavaScript



Under the hood

app.R

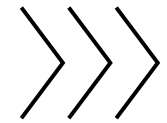
```r
library(shiny)

ui <- fluidPage(
  "Hello, ISPE! :)"
)

server <- function(input, output, session){

}

shinyApp(ui,server)
```



```html
<!DOCTYPE html> == $0
<html>
 <head> ... </head>
 <body>
  <div class="container-fluid">
   ::before
   "Hello, ISPE! :)"
   ::after
  </div>
 </body>
</html>
```

We **only** need to code in R

app.R

```
1
2  library(shiny)
3
4  ui <- fluidPage(
5    "Hello, ISPE! :)"
6  )
7
8  server <- function(input, output, session){
9
10 }
11
12 shinyApp(ui,server)
13
```

Server function has 3 arguments: input, output, session

app.R

```
1
2  library(shiny)
3
4  ui <- fluidPage(
5    "Hello, ISPE! :)"
6  )
7
8  server <- function(input, output, session){
9
10 }
11
12 shinyApp(ui,server)
13
```

shinyApp() function builds the ui and server together.

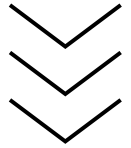
Input controls

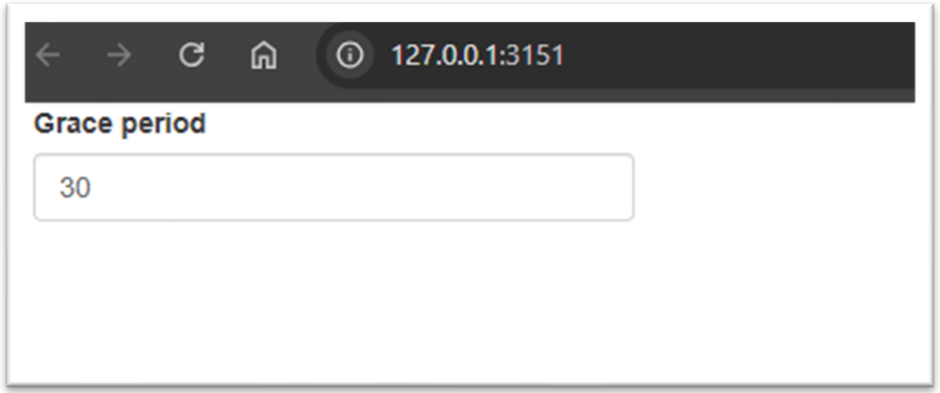
For users to interact with the app we need to give them **input controls**

Input controls have 3 essential arguments:

- **inputId**
- **label**
- **value**

```
21  ui <- fluidPage(
22
23    shiny::numericInput(
24      inputId = "gracePeriod",
25      label = "Grace period",
26      value = 30
27    )
28
29  )
```





File: 03_app.R in GitHub repository

Reactive outputs

Shiny has functions that display R objects in the UI

These functions are called ***Output** family

Specific for the output type

UI output function	**creates**	**via a matching render function in the server**
textOutput	Text	renderText
plotOutput	Plot	renderPlot
uiOutput	Raw HTML	renderUI
tableOutput	Table	renderTable
dataTableOutput	DataTable	renderDataTable

2 step process

Step 1

Tell Shiny what & how to build the object in the server function

```
37
38  library(shiny)
39
40  ui <- fluidPage(
41
42    shiny::numericInput(
43      inputId = "gracePeriod",
44      label = "Grace period",
45      value = 30
46    ),
47
48    textOutput(outputId = "gracePeriodNote")
49
50  )
51
52  server <- function(input, output, session){
53
54    output$gracePeriodNote <- renderText({
55
56      paste("The grace period is:", input$gracePeriod)
57
58    })
59
60  }
61
62  shinyApp(ui,server)
63
```

File: 03_app.R in GitHub repository

2 step process

Step 1

Tell Shiny what & how to build the object in the server function

```
37
38  library(shiny)
39
40  ui <- fluidPage(
41
42    shiny::numericInput(
43      inputId = "gracePeriod",
44      label = "Grace period",
45      value = 30
46    ),
47
48    textOutput(outputId = "gracePeriodNote")
49
50  )
51
52  server <- function(input, output, session){
53
54    output$gracePeriodNote <- renderText({
55
56      paste("The grace period is:", input$gracePeriod)
57
58    })
59
60  }
61
62  shinyApp(ui,server)
63
```

Input object from the UI that the server can access

File: 03_app.R in GitHub repository

2 step process

Step 1

Tell Shiny what & how to build the object in the server function

```
37
38  library(shiny)
39
40  ui <- fluidPage(
41
42    shiny::numericInput(
43      inputId = "gracePeriod",
44      label = "Grace period",
45      value = 30
46    ),
47
48    textOutput(outputId = "gracePeriodNote")
49
50  )
51
52  server <- function(input, output, session){
53
54    output$gracePeriodNote <- renderText({
55
56      paste("The grace period is:", input$gracePeriod)
57
58    })
59
60  }
61
62  shinyApp(ui,server)
63
```

Important!
outputId must be unique

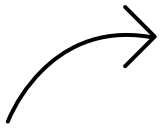
Input object from the UI that the server can access

File: 03_app.R in GitHub repository

2 step process

Step 2

Tell Shiny where to display the objects



```
37
38  library(shiny)
39
40  ui <- fluidPage(
41
42    shiny::numericInput(
43      inputId = "gracePeriod",
44      label = "Grace period",
45      value = 30
46    ),
47
48    textOutput(outputId = "gracePeriodNote")
49
50  )
51
52  server <- function(input, output, session){
53
54    output$gracePeriodNote <- renderText({
55
56      paste("The grace period is:", input$gracePeriod)
57
58    })
59
60  }
61
62  shinyApp(ui,server)
63
```

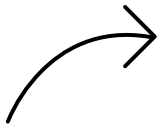
Important!
outputId must match in UI

File: 03_app.R in GitHub repository

2 step process

Step 2

Tell Shiny where to display the objects



```
37
38  library(shiny)
39
40  ui <- fluidPage(
41
42    shiny::numericInput(
43      inputId = "gracePeriod",
44      label = "Grace period",
45      value = 30
46    ),
47
48    textOutput(outputId = "gracePeriodNote")
49
50  )
51
52  server <- function(input, output, session){
53
54    output$gracePeriodNote <- renderText({
55
56      paste("The grace period is:", input$gracePeriod)
57
58    })
59
60  }
61
62  shinyApp(ui,server)
63
```

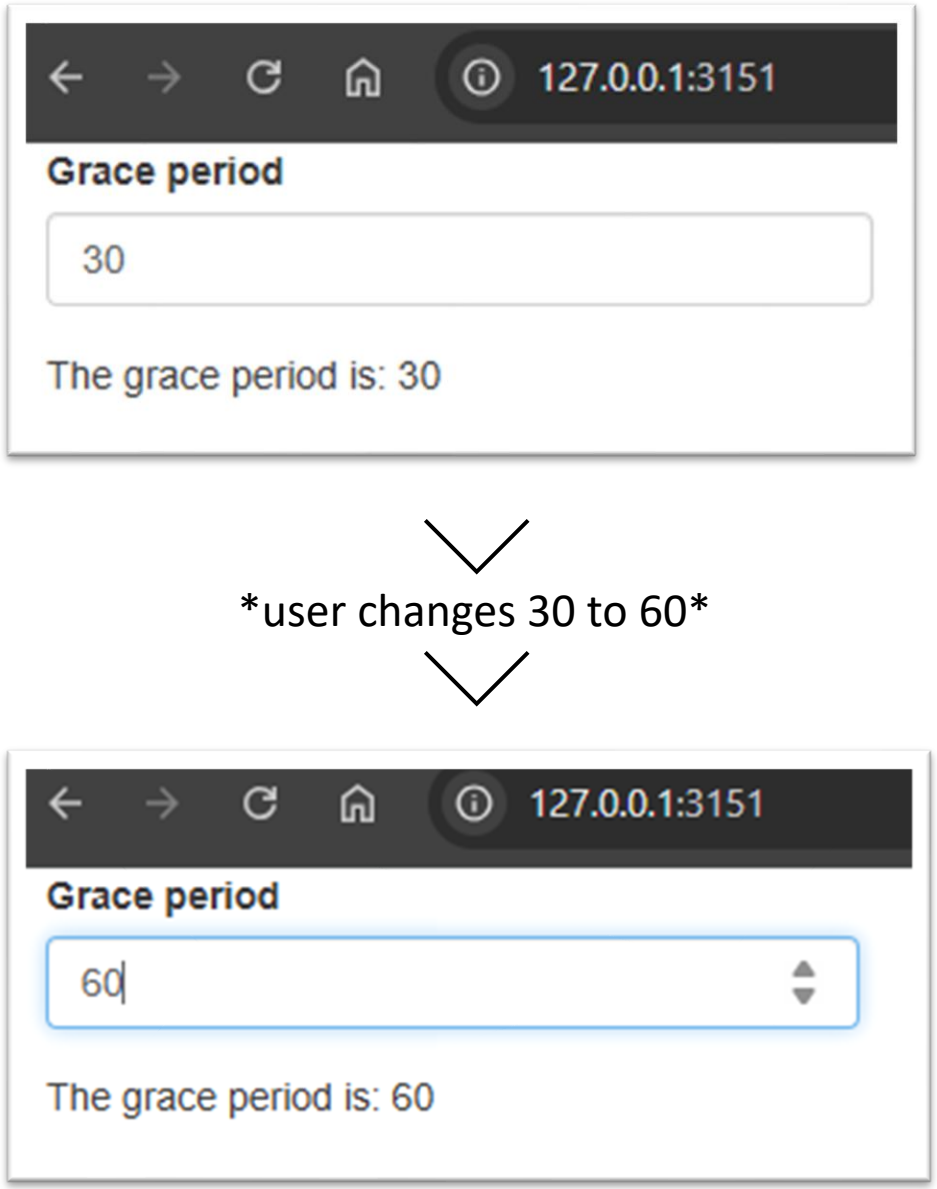
File: 03_app.R in GitHub repository

Reactive outputs

User interaction in the UI makes the HTML **react** to changes.

The server part of the app uses **reactive programming**

The outputs are **reactive outputs**



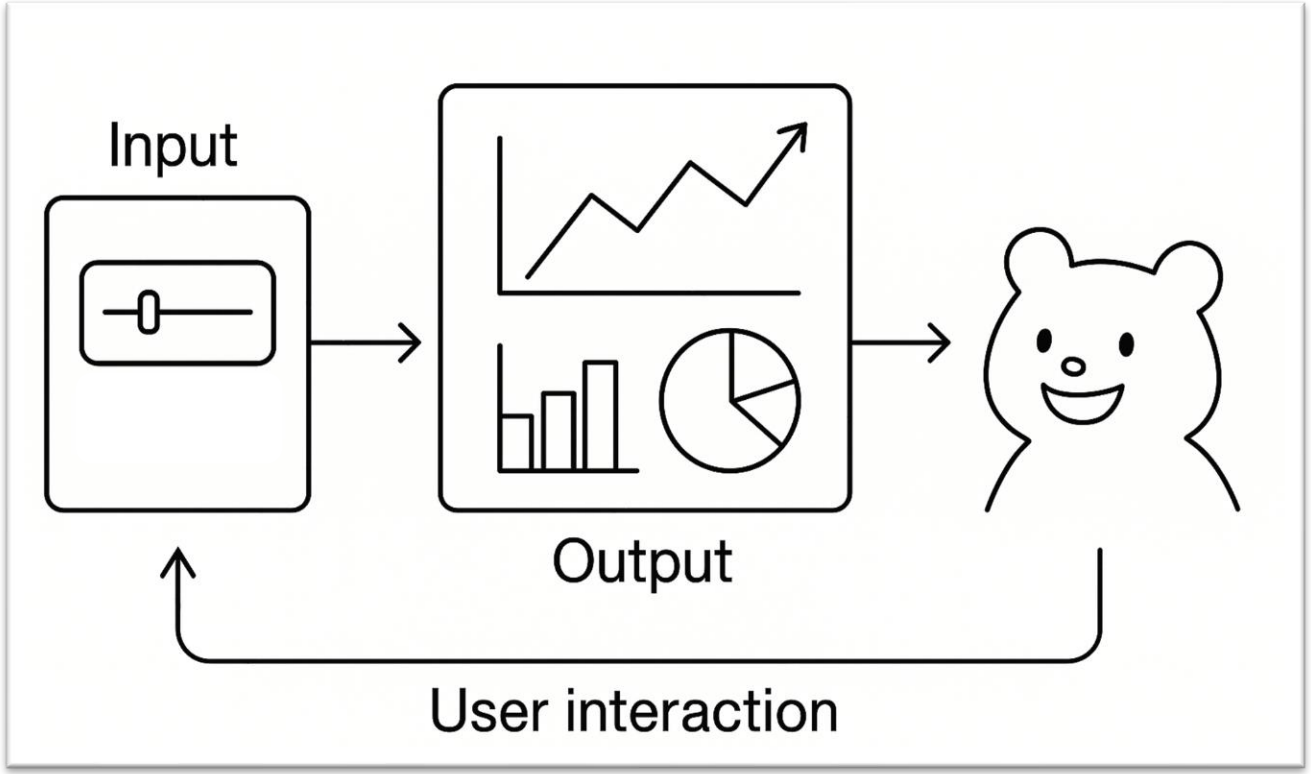
File: 04_app.R in GitHub repository

A reactive output is eager

Immediate feedback: the output updates automatically as soon as the user changes an input

No manual refresh needed, it just works

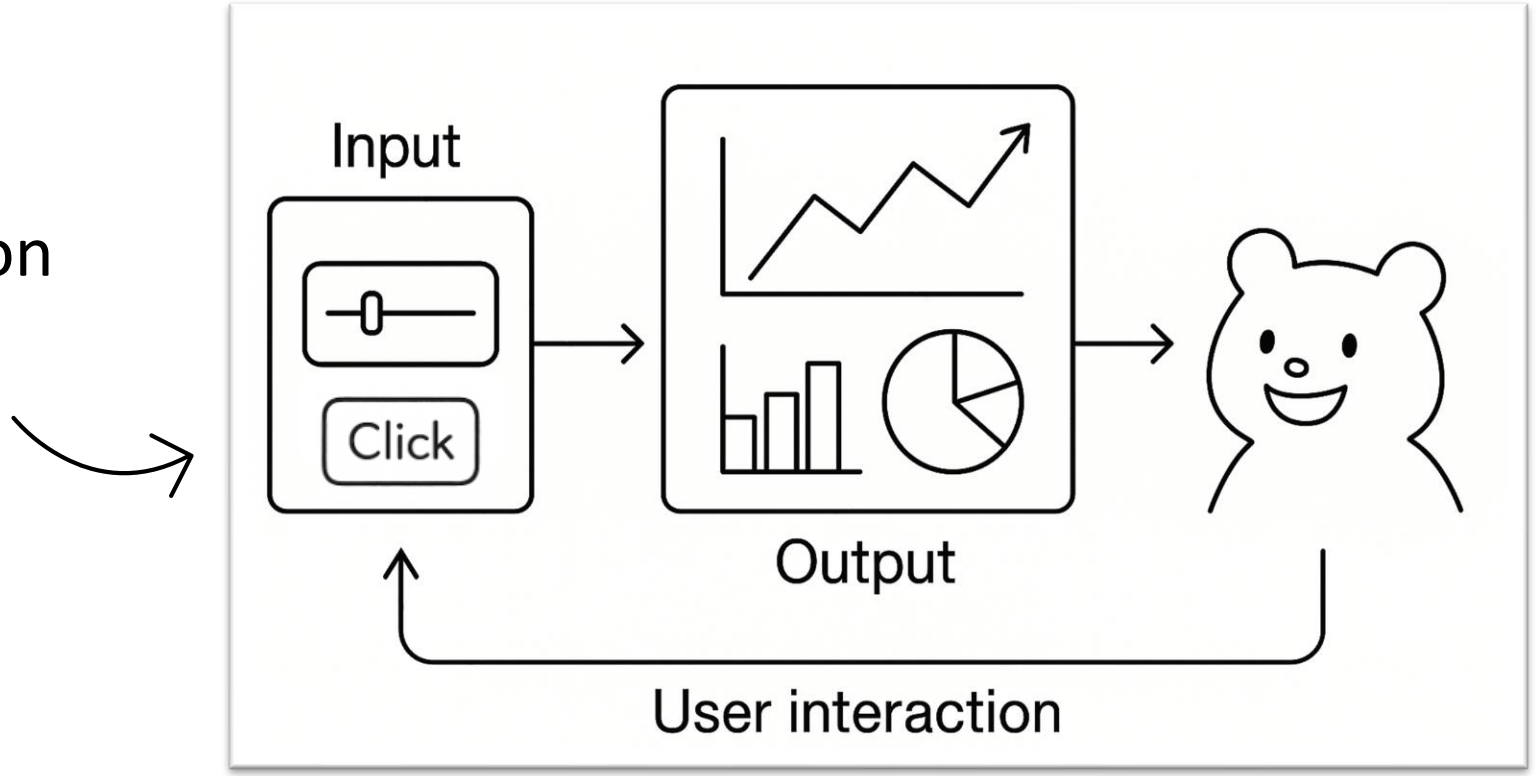
Can be limiting when you want to adjust multiple inputs before recalculating a resource-intensive output



A reactive output is eager

BUT

We can trigger a calculation only when we want to, using an **actionButton()**



actionButton() in the UI with an id “button”


```
65  library(shiny)
66
67  ui <- fluidPage(
68
69    shiny::numericInput(
70      inputId = "gracePeriod",
71      label = "Grace period",
72      value = 30),
73
74    actionButton(inputId = "button",
75                 label = "Click me"),
76
77    textOutput(outputId = "gracePeriodNote")
78  )
79
80  server <- function(input, output, session){
81
82    output$gracePeriodNote <- renderText({
83      paste("The grace period is:", input$gracePeriod)
84    }) |> bindEvent(input$button)
85
86  }
87
88  shinyApp(ui,server)
89
```

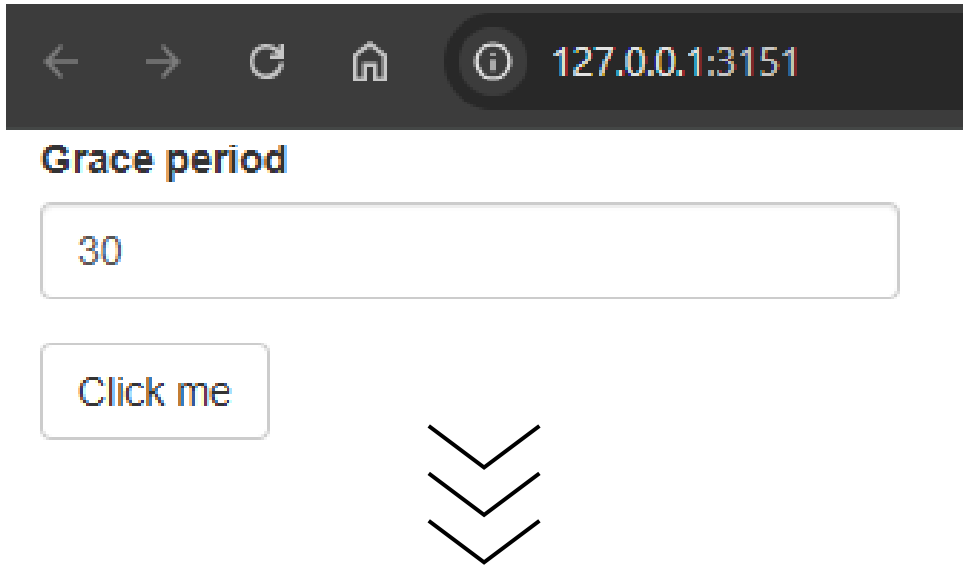
File: 04_app.R in GitHub repository

actionButton() in the UI with an id “button”

bindEvent() to only trigger when the user clicks the button

```
65  library(shiny)
66
67  ui <- fluidPage(
68
69    shiny::numericInput(
70      inputId = "gracePeriod",
71      label = "Grace period",
72      value = 30),
73
74    actionButton(inputId = "button",
75                 label = "Click me"),
76
77    textOutput(outputId = "gracePeriodNote")
78  )
79
80  server <- function(input, output, session){
81
82    output$gracePeriodNote <- renderText({
83      paste("The grace period is:", input$gracePeriod)
84    }) |> bindEvent(input$button)
85
86  }
87
88  shinyApp(ui,server)
89
```

File: 04_app.R in GitHub repository


127.0.0.1:3151
Grace period
30
Click me
127.0.0.1:3151
Grace period
30
Click me
The grace period is: 30

We create a recipe, Shiny cooks it!



The code doesn't ***tell*** Shiny to create the output and send it to the browser, but it informs ***how to*** create the output if it needs to

Imperative programming



You tell the computer **what, how & when to do**

VS

Declarative programming



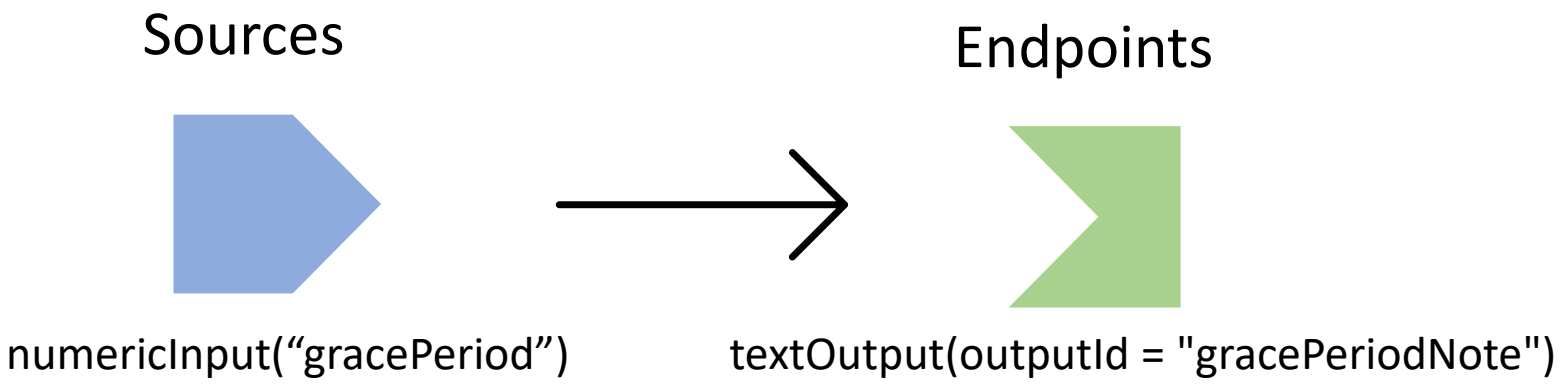
You describe **what you want, inputs and outputs**. Shiny figures out when and how to do it

The reactive graph

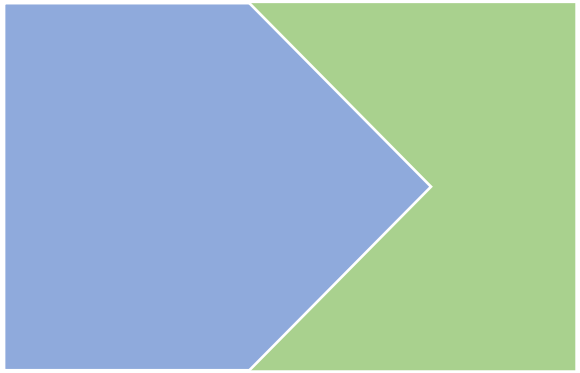
A **visual map** showing how inputs, reactive expressions, and outputs are connected

Helps understanding **what triggers what** in your Shiny app

Shiny is lazy: code runs only when needed, **NOT** top-to-bottom



Sources can affect **Endpoints** directly



Reactive expression

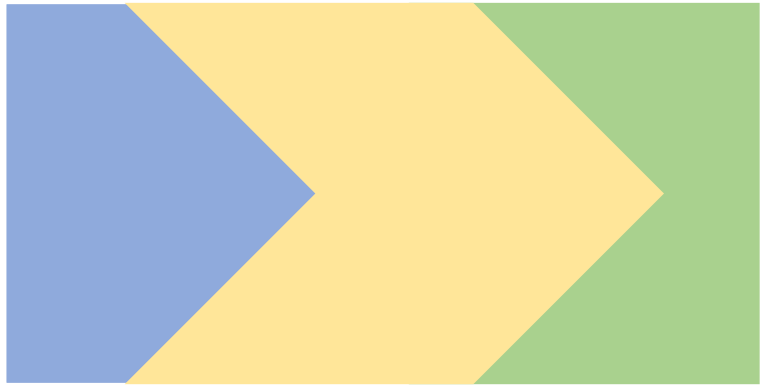
Reduces duplication in your reactive code by introducing additional nodes into the reactive graph

Function **eventReactive()** only runs when button is clicked and stores the text in **note()**

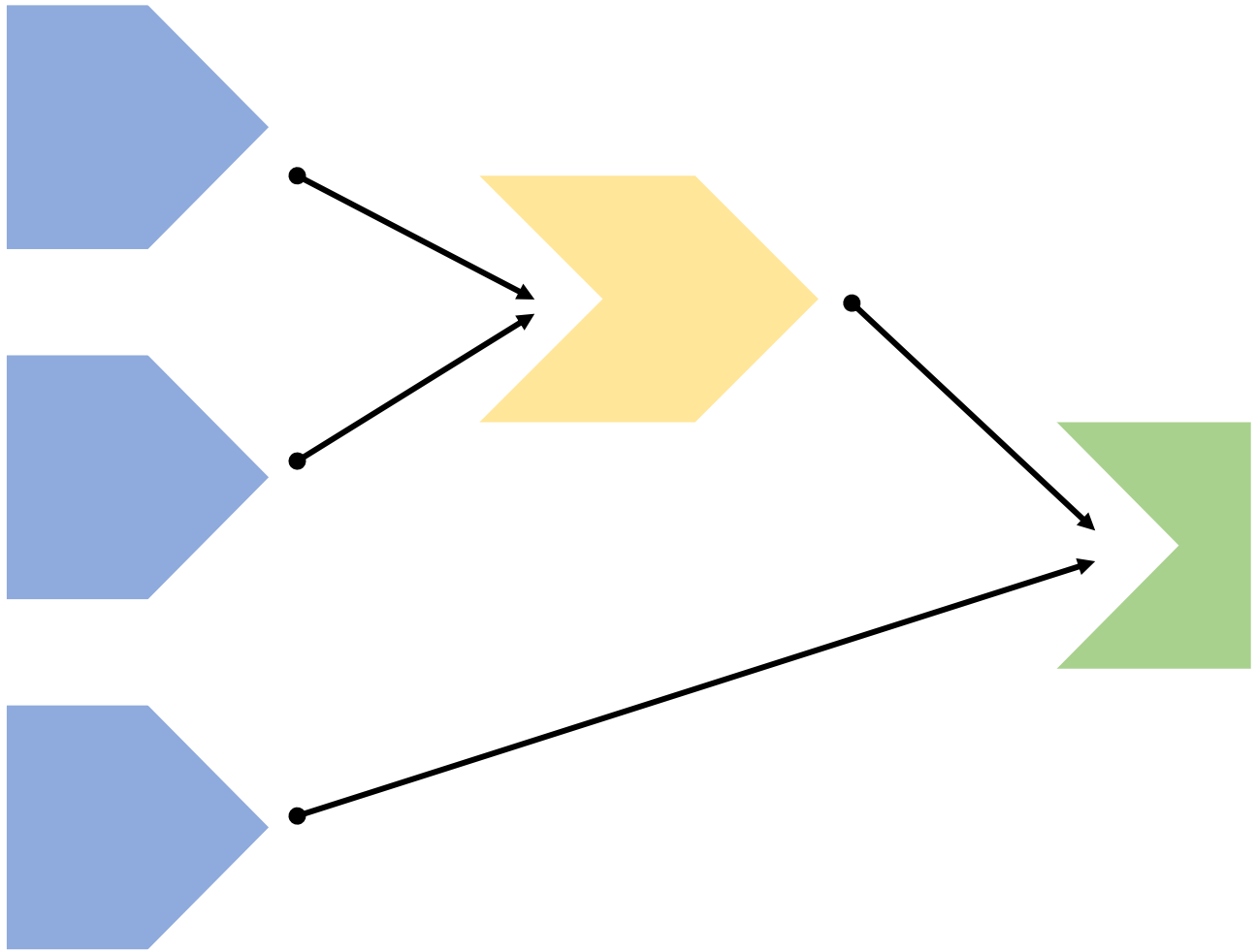
```
120 library(shiny)
121
122 ui <- fluidPage(
123
124   numericInput(
125     inputId = "gracePeriod",
126     label   = "Grace period",
127     value   = 30),
128
129   actionButton(
130     inputId = "button",
131     label   = "Click me"),
132
133   textOutput(outputId = "gracePeriodNote")
134 )
135
136 server <- function(input, output, session){
137
138    note <- eventReactive(input$button, {
139      paste("The grace period is:", input$gracePeriod)
140   })
141
142   output$gracePeriodNote <- renderText({
143     note()
144   })
145 }
146
```

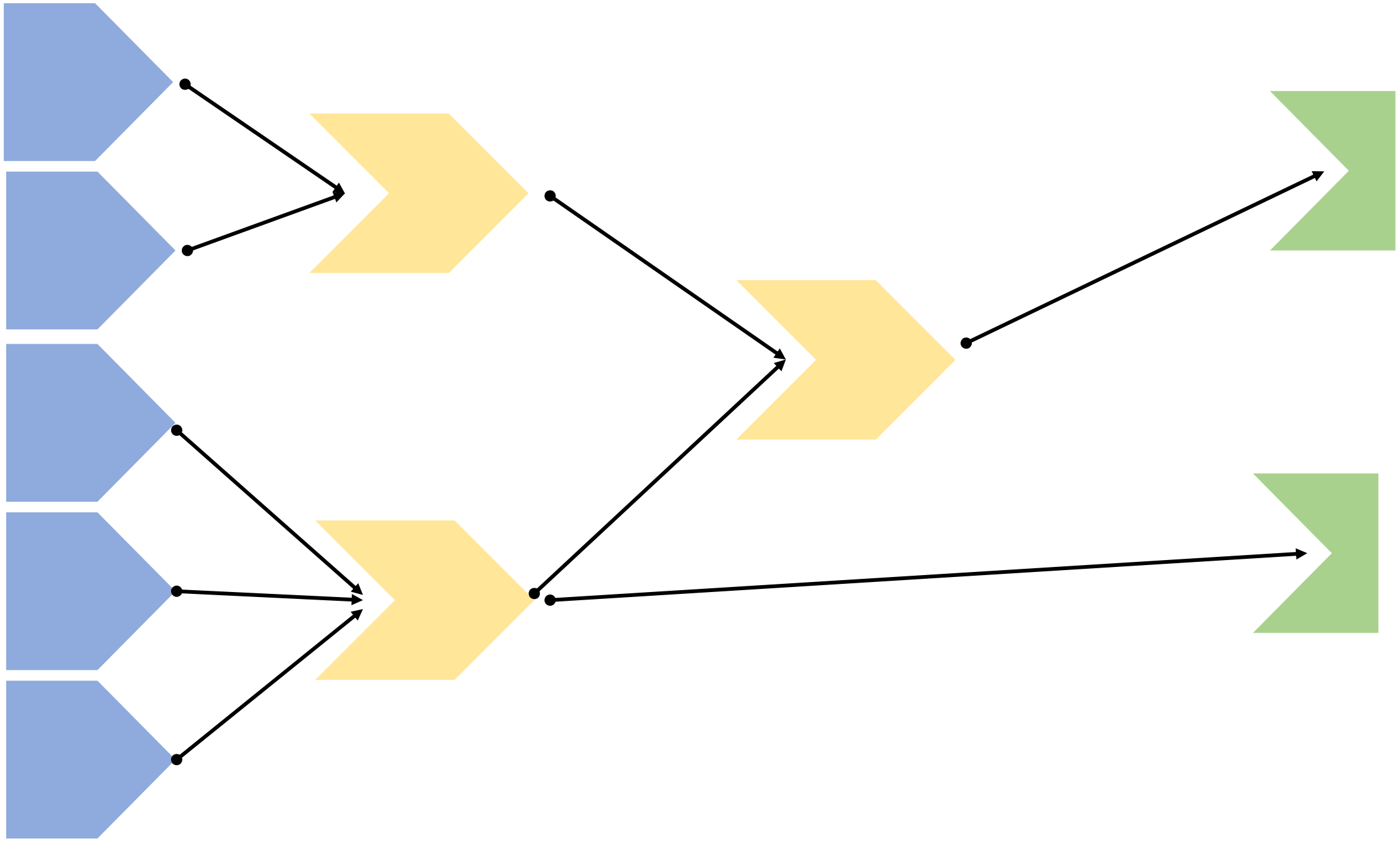
File: 05_app.R in GitHub repository

Sources affects Conductor that affects Endpoints



Reactive expression is a conductor





Let's implement the data prep into the app

```
server <- function(input, output, session) {
  #Generate data once
  data <- generate_data()

  # Reactive preparation of treatment episode data
  prep_data <- reactive({

    #aggregate by dispensation
    agg <- data |>
      dplyr::arrange(person_id, drug, disp_date) |>
      dplyr::group_by(person_id, drug, disp_date) |>
      dplyr::summarize(
        total_ddd = base::sum(ddd_pack * num_pack),
        total_tab = base::sum(units_pack * num_pack),
        sex       = dplyr::first(sex),
        age_group = dplyr::first(age_group),
        .groups   = "drop"
      ) |>
      dplyr::mutate(
        days_supply = total_tab / 2,
        end_date    = disp_date + base::ceiling(days_supply) + lubridate::days(input$gracePeriod)
      )
```

We create a reactive dataset with the **reactive()** function

File: 06_app.R in GitHub repository

Let's implement the data prep into the app

```
server <- function(input, output, session) {
  #Generate data once
  data <- generate_data()

  # Reactive preparation of treatment episode data
  prep_data <- reactive({

    #aggregate by dispensation
    agg <- data |>
      dplyr::arrange(person_id, drug, disp_date) |>
      dplyr::group_by(person_id, drug, disp_date) |>
      dplyr::summarize(
        total_ddd = base::sum(ddd_pack * num_pack),
        total_tab = base::sum(units_pack * num_pack),
        sex       = dplyr::first(sex),
        age_group = dplyr::first(age_group),
        .groups   = "drop"
      ) |>
      dplyr::mutate(
        days_supply = total_tab / 2,
        end_date    = disp_date + base::ceiling(days_supply) + lubridate::days(input$gracePeriod)
      )
```

We include the **input$gracePeriod** that we defined in the UI.

File: 06_app.R in GitHub repository

Create reactive output in the server

```
157
158   #Kaplan-Meier plot
159   output$survPlot <- renderPlot({
160     df <- prep_data()
161
162     #Surv formula + fit
163     fit <- "Surv(time_months, disc) ~ drug" |>
164       stats::as.formula() |>
165       survminer::surv_fit(data = df)
166
167     # generate survival plot
168     survminer::ggsurvplot(
169       fit,
170       data       = df,
171       risk.table = TRUE,
172       pval       = TRUE,
173       conf.int   = TRUE,
174       ggtheme    = ggplot2::theme_minimal(),
175       xlab       = "Months since start",
176       ylab       = "Persistence (%)"
177     )
178
179   })
```

We create a **reactive output** called survPlot that we can **add to the UI**.

File: 06_app.R in GitHub repository

Create reactive output in the server

```
157
158   #Kaplan-Meier plot
159   output$survPlot <- renderPlot({
160     df <- prep_data()
161
162     #Surv formula + fit
163     fit <- "Surv(time_months, disc) ~ drug" |>
164       stats::as.formula() |>
165       survminer::surv_fit(data = df)
166
167     # generate survival plot
168     survminer::ggsurvplot(
169       fit,
170       data       = df,
171       risk.table = TRUE,
172       pval       = TRUE,
173       conf.int   = TRUE,
174       ggtheme    = ggplot2::theme_minimal(),
175       xlab       = "Months since start",
176       ylab       = "Persistence (%)"
177     )
178
179   })
```

We call the **reactive data** as a function!

File: 06_app.R in GitHub repository

Add the output to the UI

```
79
80  ui <- shiny::fluidPage(
81
82    titlePanel("Time-to-First Discontinuation Analysis"),
83
84    shiny::sidebarLayout(
85      shiny::sidebarPanel(
86          shiny::numericInput(
87            inputId = "gracePeriod",
88            label   = "Grace period (days):",
89            value   = 30,
90            min     = 0
91          )
92      ),
93      shiny::mainPanel(
94        shiny::plotOutput("survPlot")
95      )
96    )
97  )
98
```

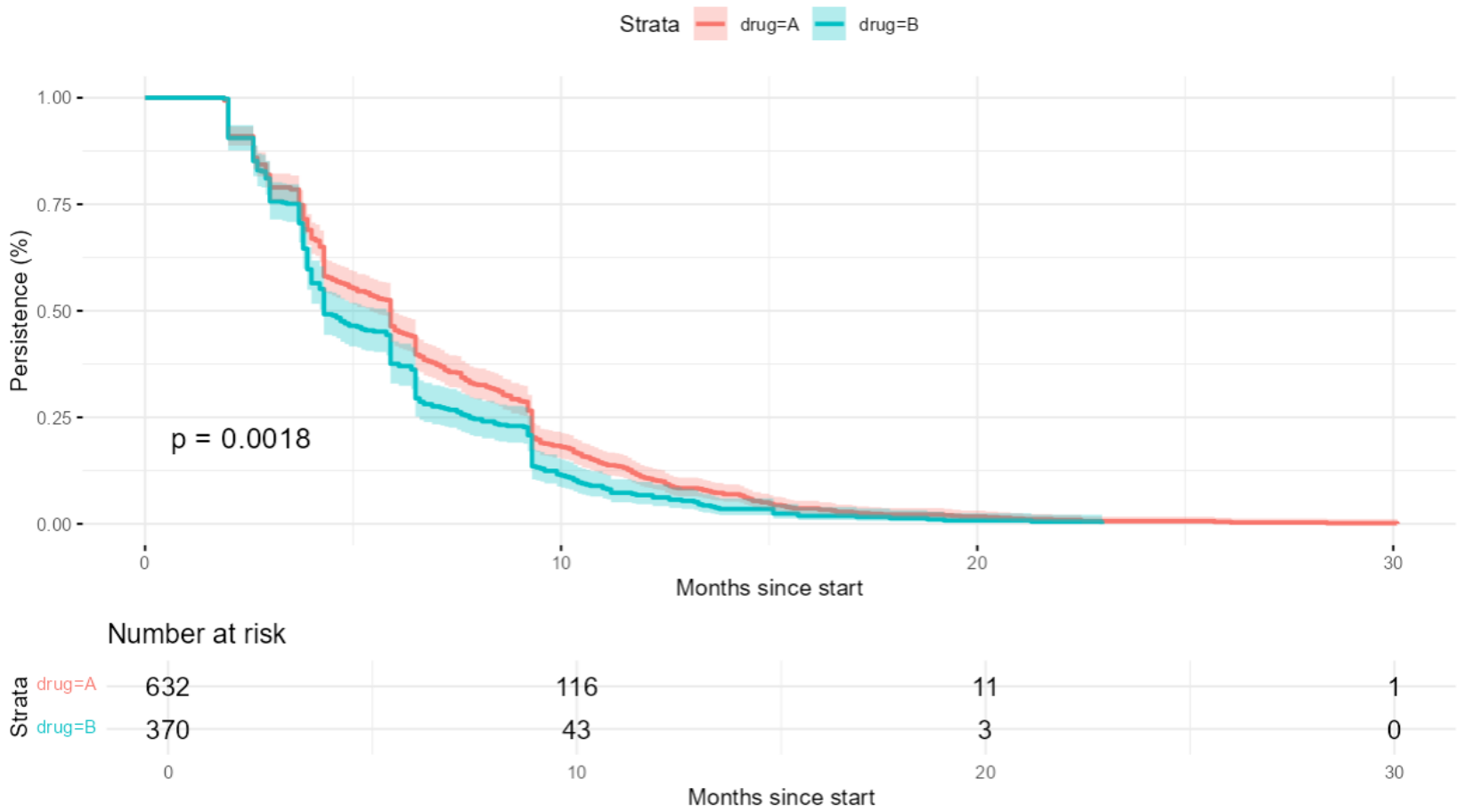
We add the **matching output** ID to the UI

File: 06_app.R in GitHub repository

Time-to-First Discontinuation Analysis

Grace period (days):

30



File: 06_app.R in GitHub repository

Discuss in pairs (3 minutes)

What other interactive elements would you add to the app?

```r

ui <- shiny::fluidPage(

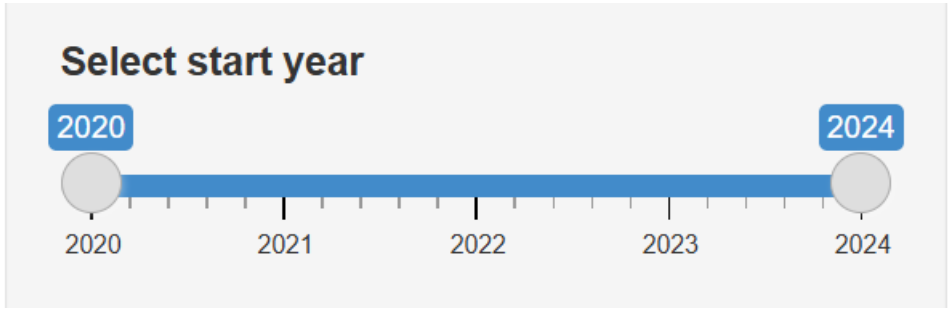
  titlePanel("Time-to-First Discontinuation Analysis"),

  shiny::sidebarLayout(
    shiny::sidebarPanel(
        shiny::numericInput(
          inputId = "gracePeriod",
          label   = "Grace period (days):",
          value   = 30,
          min     = 0
        ),
        shiny::sliderInput(
          inputId = "startYear",
          label   = "Select start year",
          min     = 2020,
          max     = 2024,
          value   = c(2020,2024),
          step    = 1,
          sep = ""
        ),
        shiny::selectInput(
          inputId = "groupVar",
          label   = "Group by:",
          choices = c(
            "None"      = "none",
            "Sex"       = "sex",
            "Age Group" = "age_group"
          )
        )

    ),
    shiny::mainPanel(
      shiny::plotOutput("survPlot")
    )
  )
)
```

Let's add a slider

sliderInput() creates a slider in the UI:



File: 07_app.R in GitHub repository

Implementation on the server side

```
157
158   # select first episodes within the date range and compute time-to-event
159   study_end <- as.Date("2024-12-31")
160   start_year_1 <- as.Date(paste0(input$startYear[1], "-01-01"))
161   start_year_2 <- as.Date(paste0(input$startYear[2], "-01-01"))
162
163   episodes <- episodes |>
164     dplyr::filter(start_date >= start_year_1 & start_date <= start_year_2) |>
165     dplyr::group_by(person_id) |>
166     dplyr::arrange(start_date) |>
167     dplyr::filter(start_date == dplyr::first(start_date)) |>
168     dplyr::ungroup() |>
169     dplyr::mutate(
170       censor_date = base::pmin(end_date, study_end),
171       disc        = dplyr::if_else(end_date < study_end, 1L, 0L),
172       time_months = round(lubridate::time_length(lubridate::interval(start_date, censor_date), unit = "month"), 1)
173     )
```

File: 07_app.R in GitHub repository

```r
ui <- shiny::fluidPage(

  titlePanel("Time-to-First Discontinuation Analysis"),

  shiny::sidebarLayout(
    shiny::sidebarPanel(
        shiny::numericInput(
          inputId = "gracePeriod",
          label   = "Grace period (days):",
          value   = 30,
          min     = 0
        ),
        shiny::sliderInput(
          inputId = "startYear",
          label   = "Select start year",
          min     = 2020,
          max     = 2024,
          value   = c(2020,2024),
          step    = 1,
          sep = ""
        ),
        shiny::selectInput(
          inputId = "groupVar",
          label   = "Group by:",
          choices = c(
            "None"      = "none",
            "Sex"       = "sex",
            "Age Group" = "age_group"
          )
        )

    ),
    shiny::mainPanel(
      shiny::plotOutput("survPlot")
    )
  )
)
```

Let's add a grouping option for the plot

selectInput() creates a dropdown selection in the UI:



File: 07_app.R in GitHub repository

Implementation on the server side

```
178
179   #Kaplan-Meier plot stratified by the selected grouping variable
180   output$survPlot <- renderPlot({
181     df <- prep_data()
182
183     if (input$groupVar == "none"){
184       fit <- "Surv(time_months, disc) ~ drug"
185     } else {
186       fit <- paste("Surv(time_months, disc) ~ drug +", input$groupVar)
187     }
188
189     fit <- fit |>
190       stats::as.formula() |>
191       survminer::surv_fit(data = df)
192
193     # generate survival plot with risk table
194     survminer::ggsurvplot(
195       fit,
196       data       = df,
197       risk.table = TRUE,
198       pval       = TRUE,
199       conf.int   = TRUE,
200       ggtheme    = ggplot2::theme_minimal(),
201       xlab       = "Months since start",
202       ylab       = "Persistence (%)"
203     )
204
205   })
```

File: 07_app.R in GitHub repository

Time-to-First Discontinuation Analysis

Grace period (days):

30

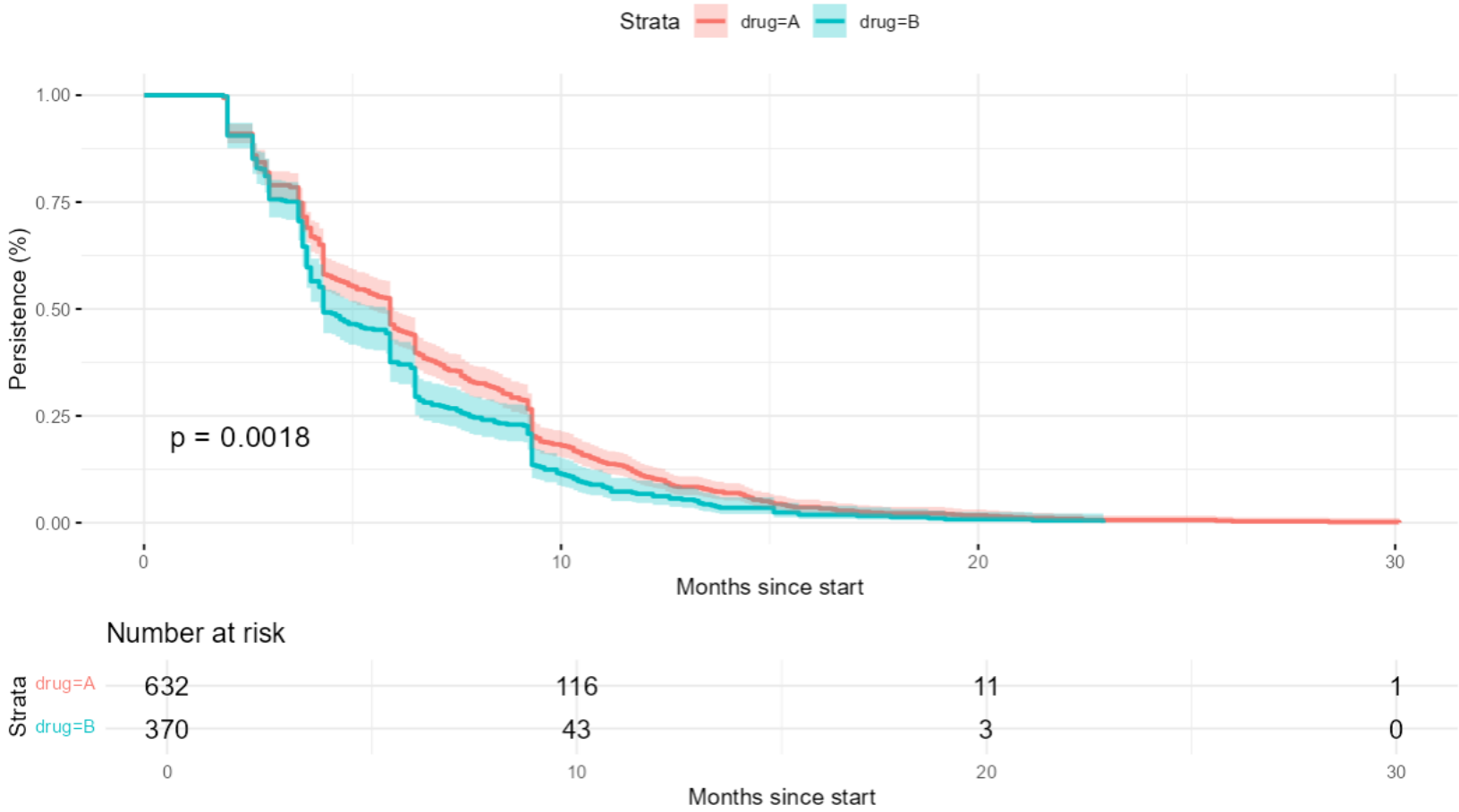
Select start year

2020 2024

2020 2021 2022 2023 2024

Group by:

None



File: 07_app.R in GitHub repository

Time-to-First Discontinuation Analysis

Grace period (days):

60

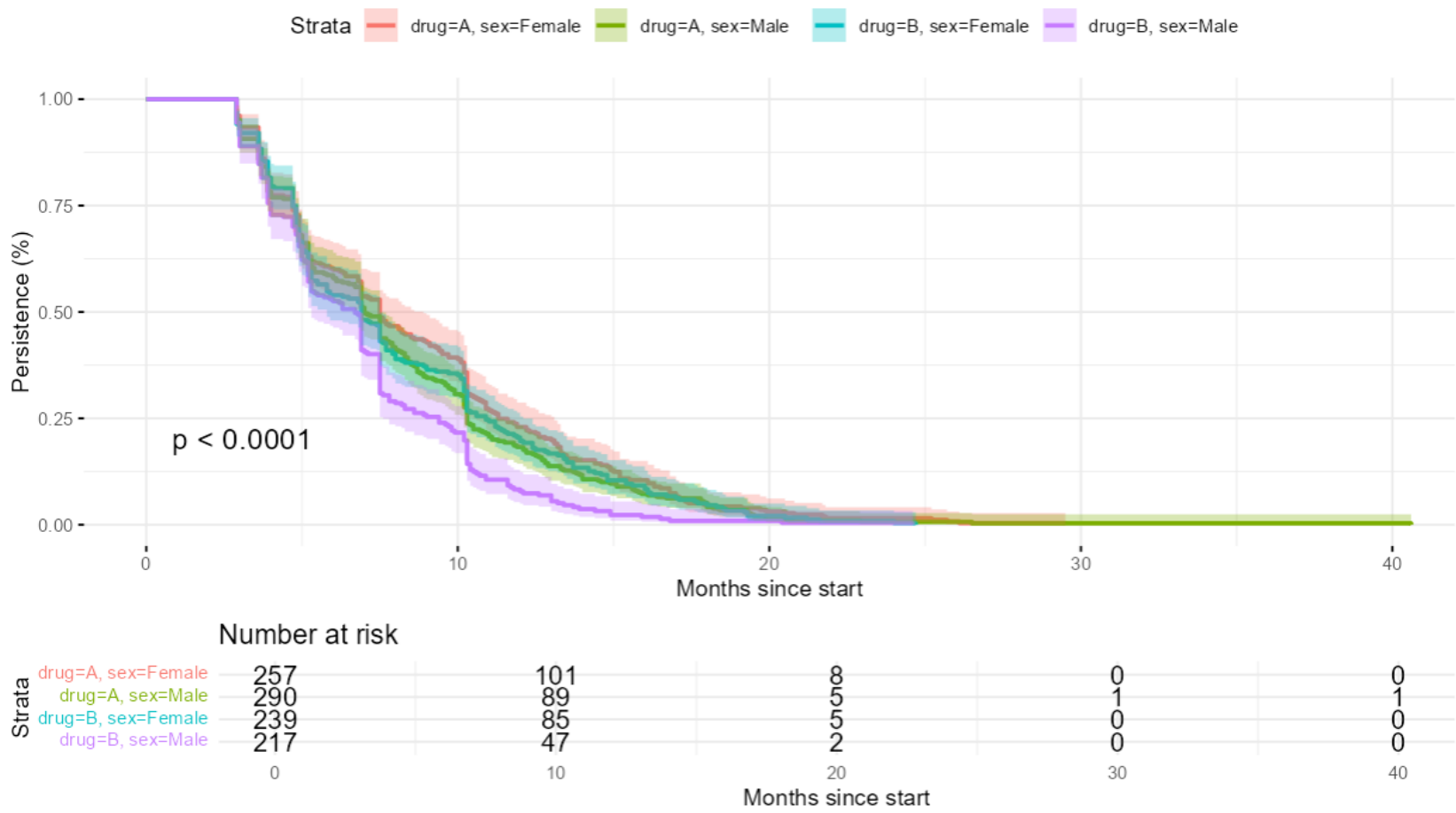
Select start year

2020 2021 2024

2020 2021 2022 2023 2024

Group by:

Sex



File: 07_app.R in GitHub repository

Discuss in pairs (3 minutes)

How would you improve the current application?

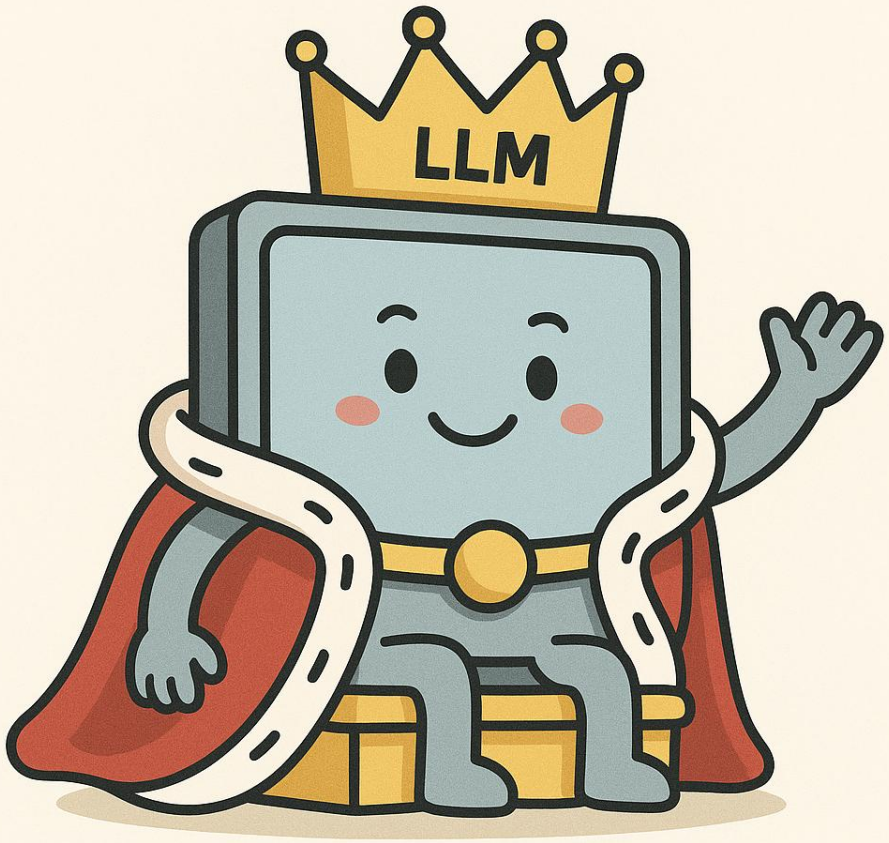
That’s up to You now!



Using LLMs to Support R Shiny App Development

Plan UI/UX Write Code

Debug & Optimize Refactor



Plan UI/UX

Brainstorm layout, user workflows, and modular structure

Write Code

Generate UI/server snippets, reactivity logic, and custom components

Debug & Optimize

Fix errors, improve performance, and clarify reactive flows

Refactor

Modularize code and improve file structure for maintainability

DEPLOYING R SHINY APPS



shinyapps.io: Easiest option; hosted by Posit (RStudio)

Posit (Rstudio) Connect: For enterprises, internal sharing with user control

ShinyProxy: open source, requires Docker-based apps

Cloud providers (e.g. AWS/Azure) Docker-based Package app with all dependencies for portable deployment

R SHINY APP SECURITY



When to worry: app contains **sensitive/restricted data**

Protect:

- Data (e.g., passwords, never hardcode, use env vars/config)
- Compute Resources (avoid letting users execute arbitrary code)

App Deployment

- IT handles server-level security
- You secure app logic

Authenticate users securely (e.g. Auth0)

Validate all user inputs



We can only predict…

R Shiny **stays strong** in pharma & life sciences

More use in **regulatory** work

Use for **regulatory-grade RWE**

Shiny is becoming **more accessible** due to LLMs

Increasing popularity among researchers and industry


Mentimeter

Thank you! Questions?



Máté Szilcz, PhD



Yuchen Guo, PhD



Jyotirmoy Sarker, PhD candidate